

Universidade do Minho
Escola de Engenharia

Laboratórios de Informática IV 2025/2026

Trabalho Prático — Grupo 35

[Imagem de grupo — Rodrigo Rocha, David Machado, Pedro Ferreira, Francisco Soares]

Francisco Miguel Fernandes Soares (A106901)
David Lopes Machado (A107325)
Pedro Ribeiro Ferreira (A107369)
Rodrigo de Sousa Campos Pacheco da Rocha (A107335)

Índice

Capítulo 1 — Conceção e Engenharia de Requisitos Assistida por LLM	4
1. Definição do Sistema	4
1.1 Contextualização	4
1.2 Motivação	5
1.3 Objetivos	7
1.4 Viabilidade	8
1.4.1 Viabilidade Financeira	8
1.4.2 Viabilidade Operacional	8
1.4.3 Viabilidade Comercial e Reputacional	8
1.5 Restrições do Sistema	9
1.5.1 Restrições Legais e de Negócio	9
1.5.2 Restrições Operacionais	9
1.5.3 Restrições Técnicas	9
1.6 Recursos e Equipa de Trabalho	10
1.6.1 Recursos Humanos	10
1.6.2 Recursos Materiais e Tecnológicos	10
1.6.3 Recursos Financeiros	10
1.7 Identificação e Análise dos Stakeholders	11
1.8 Planeamento e Cronograma do Projeto	11
2. Levantamento e Análise de Requisitos	11
2.1 Método de Levantamento e de Análise de Requisitos Adotado	11
2.1.1 Realização de Entrevistas	12
2.1.2 Survey Quantitativo (Questionários)	14
2.1.3 Relatório de Análise de Artefatos Técnicos	15
2.2 User Stories	16
2.3 Especificação dos Requisitos Levantados	17
2.3.1 Requisitos Funcionais (RF)	17
2.3.2 Requisitos Não Funcionais (RNF)	18
2.4 Análise, Validação e Refinamento de Requisitos	19
2.5 Modelação de Casos de Uso	21
2.6 Diagrama de Casos de Uso	23
Capítulo 2 — Arquitetura e Design do Software Utilizando LLM	25
3. Arquitetura Global do Sistema	25
3.1 Visão Geral e Decisões Arquiteturais	25
3.1.1 Padrão Arquitetural Adotado	25
3.1.2 Camadas Lógicas do Sistema	25
3.2 Stack Tecnológico	26
3.2.1 Backend (Lógica de Negócio e API): Python e FastAPI	26
3.2.2 Base de Dados (Persistência): MySQL	27
3.2.3 Frontend (Interface do Utilizador): Vue.js com HTML e CSS	28
4. Modelação de Domínio	28
4.1 Identificação e Caracterização das Classes do Sistema	29
4.2 Relacionamentos e Multiplicidades (Relacionamentos UML)	29
4.3 Especificação dos Atributos das Classes	31

4.4 Diagrama de Classes	34
4.5 Consolidação e Revisão do Diagrama de Classes	35
5. Modelação Dinâmica e Componentes	35
5.1 Diagrama de Componentes	36
5.2 Diagramas de Sequência	36
6. Interfaces e Comunicação	37
6.1 Especificação Preliminar da API REST	37
6.2 Design de Interfaces e Wireframes Textuais	38
Capítulo 3 — Implementação do Sistema	40
7. Implementação do Backend Aplicacional	40
7.1 Organização e Abordagem de Desenvolvimento	40
7.2 Contrato da API	40
7.3 Schemas e Validações	41
7.4 Autenticação e Controlo de Acesso	41
7.5 Módulos Funcionais	41
7.6 Máquina de Estados das Ordens de Serviço	42
7.7 Regras de Negócio	43
7.8 Funcionalidades Auxiliares	43

Capítulo 1 — Conceção e Engenharia de Requisitos Assistida por LLM

1. Definição do Sistema

1.1 Contextualização

A faísca que deu origem à DLMCare não surgiu numa sala de reuniões, mas sim na rotina acelerada e, por vezes, caótica da mobilidade urbana portuguesa. David Lopes Machado, um técnico de eletromecânica apaixonado por soluções de transporte sustentáveis, usava a sua própria trotinete elétrica diariamente para fugir ao trânsito.

Tudo começou quando a sua trotinete avariou. Ao procurar uma solução, David deparou-se com um mercado incipiente e frustrante: as poucas oficinas existentes demoravam semanas a dar resposta, não tinham peças de substituição e, muitas vezes, devolviam os equipamentos com problemas mal resolvidos. Foi então que decidiu deitar mãos à obra e consertar o veículo na sua própria garagem. O que começou como uma necessidade pessoal rapidamente se transformou num “boca a boca” entre amigos e colegas que partilhavam da mesma dor. Numa questão de meses, a garagem de David estava repleta de trotinetes à espera de reparação.

Foi nesse momento de rutura — entre o espaço exíguo da sua casa e a procura avassaladora — que o “clique” empresarial aconteceu. David percebeu que a micromobilidade não era apenas uma moda passageira, mas o futuro do planeamento urbano. As trotinetes multiplicavam-se nas ruas, mas a infraestrutura de suporte era praticamente inexistente. Havia ali uma lacuna de mercado gritante e uma oportunidade de negócio clara.

Com as economias que juntou e um plano de negócios focado na rapidez e na transparência, David deu o salto. Nasceu assim, oficialmente, a DLMCare. Para garantir visibilidade e conveniência, escolheu uma localização estratégica, abrindo a loja principal na Avenida Almirante Reis, n.º 74B, em Lisboa — uma das artérias com maior fluxo de ciclovias e utilizadores de mobilidade suave da capital.

Rapidamente, David percebeu que não podia fazer tudo sozinho. Para que ele se pudesse focar na expansão e nos casos mais complexos, montou a sua primeira equipa de confiança:

- **José Barros (Gerente de Loja):** O maestro dos bastidores e o guardião do stock. Com um olho clínico para a logística e um talento natural para otimizar processos, atua como o motor invisível da oficina.
- **Inês Carvalho (Rececionista):** A primeira cara da DLMCare. Com uma paciência inesgotável e um talento natural para a organização, ficou encarregue de domar o caos diário.
- **Tiago Mendes (Mecânico Especialista em Eletrónica):** O “MacGyver” das baterias e das motherboards, contratado para diagnosticar as falhas elétricas mais crónicas.
- **João “Ruca” Silva (Mecânico de Estrutura e Hidráulica):** Especialista em travões, afinações de chassis e pneus.

O método DLMCare foi um sucesso tão estrondoso que, em menos de dois anos, começaram a receber clientes de fora da Grande Lisboa. Iniciou-se assim o plano de expansão, com duas novas filiais estratégicas:

- **DLMCare Porto:** Localizada na movimentada Rua de Júlio Dinis, perto da zona da Boavista, a cargo de Miguel Torres.
- **DLMCare Braga:** Situada na Avenida da Liberdade, no coração da cidade, entregue a Sofia Almeida.

Hoje, a DLMCare já não é apenas uma oficina de bairro; é uma cadeia de referência na reparação de trotinetes elétricas. No entanto, gerir o stock, os clientes e a faturação de três lojas movimentadas em cidades diferentes apenas com folhas de Excel e telefonemas revelou-se o próximo grande desafio.

Prompt

Contexto Inicial: Atua como um Analista de Requisitos com talento para storytelling empresarial. Estou a desenvolver um sistema para a gestão de uma oficina de trotinetes (DLMCare). Cria a história do fundador, David Lopes Machado. Conta como ele começou, descreve o momento em que percebe a oportunidade de negócio, relata a abertura da sua oficina oficial “DLMCare” (com morada realista), menciona a contratação dos primeiros funcionários. Objetivo: texto fluido, realista e com um tom narrativo pessoal semelhante a um caso de estudo de empreendedorismo.

Análise Crítica

Ao analisar a resposta à prompt ficámos bastante satisfeitos, contudo pedimos para adicionar mais algumas lojas noutras cidades e para acrescentar um gestor de loja à inicial, pois achamos que apenas uma loja era demasiado pouco e fazia sentido até a primeira loja ter um gerente.

Modelo: Gemini

1.2 Motivação

A expansão da DLMCare para o Porto e Braga foi um marco de sucesso, mas gerir três lojas movimentadas com os processos manuais da loja original transformou o sonho de David Lopes Machado num verdadeiro pesadelo logístico. O modelo de negócio estava a escalar muito mais rápido do que a sua infraestrutura de gestão, e as fissuras começaram a expor-se sob a pressão de centenas de clientes semanais.

O Caos Descentralizado e as Falhas de Comunicação

O sistema inicial baseado em cadernos, folhas impressas e post-its já era mau numa só loja; em três, tornou-se insustentável. O dia a dia das equipas transformou-se numa gestão de crises constante através de grupos de WhatsApp caóticos:

- **Perda de Histórico entre Lojas:** Um cliente que comprou ou reparou a sua trotinete em Lisboa (sendo atendido pela Inês), e que mais tarde se mudou para Braga, chegava à loja da Sofia e era tratado como um cliente desconhecido. Não havia histórico de reparações, garantias ou intervenções anteriores.
- **Telefone Estragado:** Quando um cliente ligava para a linha geral a perguntar pelo estado da sua trotinete, a pessoa que atendia tinha frequentemente de enviar mensagens para o grupo de WhatsApp “Mecânicos DLMCare” a perguntar “Malta, de quem é a Ninebot amarela do Sr. Costa?”

Está em Lisboa ou no Porto?”. As respostas demoravam horas, gerando frustração do outro lado da linha.

- **Ordens de Serviço Perdidas:** Folhas de obra em papel continuavam a sujar-se de óleo ou a perder-se nas bancadas. Para piorar, o “padrão DLMCare” começou a falhar: o Miguel no Porto descrevia as avarias de uma forma, enquanto a equipa de Lisboa usava outra terminologia, dificultando a análise dos problemas mais comuns.

O Pesadelo Logístico das Peças e o Ponto Cego Financeiro

Na retaguarda, a ausência de um sistema em rede estava a custar muito dinheiro à empresa. O armazém operava com base em folhas de Excel partilhadas que raramente eram atualizadas em tempo real:

- **Stock Desequilibrado e Imobilização:** Faltavam frequentemente peças críticas em Lisboa (como controladores Xiaomi), levando o Tiago a imobilizar trotinetes durante semanas à espera de uma encomenda do fornecedor. O absurdo? O Miguel, no Porto, tinha cinco desses controladores a ganhar pó na prateleira, mas como ninguém atualizava o Excel, o David gastava dinheiro em novas encomendas desnecessárias em vez de transferir o stock internamente.
- **Gestão Financeira por “Palpite”:** No final do mês, o David passava noites em claro a tentar consolidar dados de três lojas diferentes. Era impossível ter uma visão clara de qual filial era mais rentável, comparar a eficiência dos mecânicos de Braga com os de Lisboa, ou sequer calcular o lucro líquido real da rede após subtrair os custos das transferências de peças e a mão de obra.

O Ponto de Viragem

A gota de água surgiu quando a DLMCare começou a receber avaliações negativas cruzadas no Google. Clientes queixavam-se de que a experiência no Porto não tinha nada a ver com a rapidez prometida pela marca em Lisboa, ou que as trotinetes ficavam retidas um mês à espera de peças comuns.

O David percebeu, com grande clareza, que o problema não estava na qualidade das suas equipas — a Inês, o Miguel, a Sofia e os mecânicos trabalhavam até à exaustão —, mas sim na ausência de um sistema central. A marca estava a sangrar dinheiro em peças duplicadas e a perder a confiança dos clientes.

Consciente de que soluções de mercado genéricas não suportavam a dinâmica ágil da micromobilidade distribuída, o David tomou uma decisão estratégica. Contactou um grupo talentoso de estudantes de Engenharia Informática, lançando-lhes um desafio: desenvolver um sistema de gestão centralizado. O sistema DLMCare teria de unificar as três lojas em tempo real, acabar com o caos do papel, automatizar as transferências de inventário e devolver ao David o controlo total sobre o império que estava a construir.

Prompt

Continua a narrativa: tendo em conta o sucesso e o volume massivo de clientes, descreve os problemas operacionais graves que começaram a surgir, usando exemplos práticos e caóticos do dia-a-dia. Menciona o caos de ter informações importantes em papel e o descontrolo de peças e custos. Conclui com o David a perceber que a desorganização está a prejudicar a reputação e a fazê-lo perder dinheiro, decidindo recorrer a um grupo de estudantes de Engenharia Informática.

Análise Crítica

Esta prompt foi enviada logo a seguir à contextualização, e após reflexão em grupo decidimos aceitar o output obtido, considerando que a motivação estava bem estruturada.

Modelo: Gemini

1.3 Objetivos

Na primeira reunião de levantamento de requisitos entre o David, os gerentes das filiais e o grupo de estudantes de Engenharia Informática, o caos operacional foi literalmente colocado em cima da mesa. Através de um mapeamento exaustivo dos processos atuais, a equipa técnica traduziu as frustrações do negócio em metas funcionais e arquitetónicas claras. Ficou imediatamente decidido que a solução não poderia ser local; teria de ser uma plataforma para sincronizar toda a operação. Definiram-se os seguintes objetivos técnicos e de negócio:

- **Registo Centralizado de Clientes e Equipamentos:** Estabelecer uma base de dados relacional para o cadastro de clientes e do respetivo parque de trotinetes (associando marca, modelo e número de série a cada proprietário), garantindo identificação unívoca e eliminando o risco de trocas ou perda de dados.
- **Gestão Integral de Ordens de Serviço (OS):** Implementar um módulo central para a criação e monitorização do ciclo de vida das OS. Este módulo deve permitir registar o diagnóstico inicial, atualizar o estado da reparação em tempo real, registar os tempos de mão de obra despendidos por cada mecânico e associar as peças consumidas à intervenção.
- **Controlo Rigoroso e Preventivo de Inventário:** Desenvolver um mecanismo de gestão de stock dinâmico, intimamente ligado às OS. O sistema deve abater automaticamente as peças utilizadas no inventário e acionar alertas automáticos de stock mínimo para componentes críticos (baterias, pneus, pastilhas de travão, discos e controladores).
- **Cálculo Automático de Custos e Faturação:** Automatizar a componente transacional de cada reparação, cruzando o valor da mão de obra (preço do serviço) com o preço de venda das peças utilizadas, calculando automaticamente o valor final e permitindo a emissão da respetiva faturação.
- **Reporting Analítico e de Apoio à Decisão:** Incorporar um painel de controlo (dashboard) para extração de relatórios de gestão, incluindo métricas operacionais (volume de trotinetes reparadas por mês, tempos médios de reparação, rácios de eficiência por mecânico) e métricas financeiras (receitas brutas, despesas com peças, lucro líquido mensal).

Prompt

Foca-te agora nos Objetivos do sistema. Com base nos problemas relatados na secção anterior, redige os objetivos principais que o sistema DLMCare tem de cumprir para organizar a oficina, garantindo que inclui objetivos para: registo centralizado e consulta rápida de clientes e trotinetes; criação e acompanhamento de OS; gestão rigorosa do stock de peças; cálculo automático de custos e emissão de faturação; geração de relatórios operacionais e financeiros. O tom deve ser puramente técnico e executivo.

Análise Crítica

Com esta prompt obtivemos um output quase excelente, sendo apenas necessário pedir à LLM para acrescentar como se chegou a estes objetivos após a primeira reunião.

1.4 Viabilidade

A decisão de investir no desenvolvimento e implementação do sistema DLMCare não é apenas uma atualização tecnológica; é um imperativo estratégico de sobrevivência e expansão. Quando analisamos as dores atuais do negócio (Secção 1.2) em contraponto com os objetivos traçados (Secção 1.3), torna-se evidente que o software proposto deixará rapidamente de ser um custo para se transformar num dos ativos mais rentáveis da empresa. A viabilidade deste projeto sustenta-se em três pilares fundamentais que garantem um Retorno sobre o Investimento (ROI) a muito curto prazo:

1.4.1 Viabilidade Financeira

Atualmente, a DLMCare está a perder dinheiro diariamente devido a ineficiências que o sistema eliminará:

- **Otimização de Capital em Stock:** Ao implementar um inventário em rede, o David deixará de encomendar peças desnecessárias para Lisboa quando o Porto ou Braga têm excedentes. O fim das aquisições duplicadas e o alerta de ruturas de stock representam uma poupança de milhares de euros anuais.
- **Faturação Rigorosa:** Sem o registo manual em papel, nenhum serviço tabelado nem peça vendida ficará por cobrar, e nenhuma peça aplicada será esquecida no orçamento final. O cálculo automático garante a maximização da margem de lucro em cada intervenção.

1.4.2 Viabilidade Operacional

O tempo que a equipa perde atualmente a gerir o caos é tempo roubado à produção.

- **Fim do “Ruído” na Comunicação:** A substituição dos grupos de WhatsApp por um workflow centralizado e acessível devolverá dezenas de horas semanais aos gerentes, rececionistas e mecânicos. A informação sobre o estado de qualquer trotinete, em qualquer loja, estará à distância de um clique.
- **Padronização do Método DLMCare:** Com processos idênticos impostos pelo sistema em todas as filiais, a formação de novos colaboradores torna-se mais rápida e barata, permitindo que a empresa abra novas lojas no futuro (como em Coimbra ou Faro) com uma infraestrutura de gestão já testada e pronta a escalar (“plug and play”).

1.4.3 Viabilidade Comercial e Reputacional

- **Experiência de Excelência ao Cliente:** Com o acesso rápido ao histórico do cliente — independentemente da loja a que este se dirija — e a capacidade de dar respostas exatas e imediatas sobre o estado da reparação, a DLMCare recuperará a sua imagem de profissionalismo. A redução drástica dos tempos de imobilização das trotinetes traduzir-se-á num aumento direto da satisfação, gerando avaliações positivas no Google e fomentando a fidelização.

Em suma, o sistema DLMCare é altamente viável e crítico. O custo de desenvolvimento será rapidamente amortizado pelo aumento da capacidade de resposta da oficina, pela eliminação de desperdícios no inventário e pela recuperação de receitas perdidas por falhas de faturação.

Com base nos problemas apresentados na motivação e nos objetivos do sistema, escreve a justificação de viabilidade do sistema DLMCare. O tom deve ser de um business case convincente, mostrando que o software é um investimento altamente rentável.

Análise Crítica

Analisando o output obtido, como grupo chegamos à conclusão que era o que esperávamos para este ponto do relatório e decidimos não fazer alterações.

Modelo: Gemini

1.5 Restrições do Sistema

Para garantir que o desenvolvimento do sistema DLMCare seja exequível e alinhado com a realidade da infraestrutura e regulamentação vigentes, é imperativo definir as restrições arquiteturais, operacionais e legais que condicionarão o projeto.

1.5.1 Restrições Legais e de Negócio

- **Conformidade com o RGPD:** O sistema armazenará dados pessoais de clientes. É obrigatório implementar mecanismos de consentimento, anonimização e a funcionalidade de “direito ao esquecimento”.
- **Certificação de Faturação (Autoridade Tributária):** O módulo financeiro terá de estar em conformidade com as exigências da AT em Portugal, ou integrar-se com um software de faturação já certificado. A fatura ao cliente usa preços de serviço e preços de venda; os custos de compra são internos.
- **Fronteira do Sistema (Gestão de Compras Externa):** O sistema DLMCare não efetua a gestão de encomendas a fornecedores. A responsabilidade do software limita-se exclusivamente ao registo de entrada (receção) do material físico no inventário da loja.

1.5.2 Restrições Operacionais

- **Acessibilidade em Ambiente de Oficina:** A interface dos mecânicos tem de ser otimizada para utilização em tablets, com botões dimensionados para toque por utilizadores que frequentemente usam luvas, exigindo o mínimo de interações possível para alterar o estado de uma OS.
- **Disponibilidade do Sistema:** O sistema deve garantir um uptime de 99,9% durante o horário de funcionamento das oficinas (segunda a sábado, das 08h00 às 20h00).
- **Curva de Aprendizagem:** O sistema deve ser suficientemente intuitivo para que um novo colaborador consiga operar as funcionalidades básicas com uma formação não superior a 4 horas.

1.5.3 Restrições Técnicas

- **Arquitetura Baseada na Cloud:** O sistema não poderá ter instalação local. Terá de ser desenvolvido como uma aplicação Web alojada na cloud, garantindo sincronização em tempo real entre Lisboa, Porto e Braga.
- **Controlo de Concorrência e Transações (ACID):** A base de dados relacional deve gerir de forma robusta a concorrência. Se dois rececionistas tentarem reservar a última peça em simultâneo, o sistema tem de garantir a integridade transacional.
- **Design Responsivo:** O frontend deve adaptar-se a desktops (receção e administração), tablets (bancadas de trabalho) e smartphones (consultas rápidas dos gerentes).

- **Políticas de Backup:** O sistema deverá executar cópias de segurança automáticas e incrementais da base de dados diariamente, com redundância geográfica.

Prompt

Foca-te agora nas Restrições do Sistema. Redige as restrições que vão limitar ou condicionar o desenvolvimento e a operação do sistema DLMCare, dividindo-as em três categorias: Legais/Negócio, Operacionais e Técnicas. O tom deve ser formal e de engenharia, definindo limites claros para a arquitetura do software.

Análise Crítica

O output gerado foi inicialmente o que o nosso grupo esperava obter. Contudo, achamos oportuno adicionar que o sistema não efetua diretamente a gestão de encomendas com os fornecedores, devidamente adicionado nas restrições legais e de negócio.

Modelo: Gemini

1.6 Recursos e Equipa de Trabalho

Para garantir o sucesso na conceção e implementação do novo sistema de informação centralizado da DLMCare, foram mobilizados recursos em três eixos fundamentais:

1.6.1 Recursos Humanos

- **Equipa de Desenvolvimento de Software:** Uma equipa externa de consultoria e engenharia informática responsável por todo o ciclo de vida do software: levantamento de requisitos, arquitetura de dados, programação, testes e implementação final.
- **Gestão de Topo e Operações (DLMCare):** Liderada por David Lopes Machado (na qualidade de Product Owner e CEO), com apoio tático direto de José Barros, Inês Carvalho, Miguel Torres e Sofia Almeida.
- **Equipa de Validação Técnica:** Composta por Tiago Mendes e João “Ruca” Silva, cujo papel é garantir que as interfaces do chão de oficina são ergonómicas e eficientes.

1.6.2 Recursos Materiais e Tecnológicos

- **Infraestrutura Cloud e Servidores:** Alojamento da aplicação e da base de dados centralizada em serviços Cloud, garantindo alta disponibilidade e sincronização em tempo real.
- **Hardware Operacional (Lojas):** Terminais de Ponto de Venda (POS) atualizados para a receção e tablets para as bancadas da oficina.
- **Stack Tecnológico de Desenvolvimento:** Ambientes de desenvolvimento integrado, sistemas de gestão de bases de dados relacionais e ferramentas de gestão ágil de projeto.

1.6.3 Recursos Financeiros

- **Orçamento de Transição Digital:** Capital de investimento alocado pela administração da DLMCare, cobrindo os custos de desenvolvimento do software, a modernização do hardware das três lojas e o licenciamento contínuo das plataformas de alojamento.

1.7 Identificação e Análise dos Stakeholders

O ecossistema da DLMCare é composto por diversas partes interessadas. A correta identificação dos stakeholders e a compreensão das suas necessidades é crucial para garantir que o sistema entrega valor real ao negócio.

David Lopes Machado (Fundador e CEO) — Promotor do projeto e principal decisor estratégico. Interesses: visão em tempo real da rede de lojas, métricas de rentabilidade exatas, eliminação de custos com peças duplicadas.

Gestores de Loja e Apoio ao Cliente (José Barros, Inês Carvalho, Miguel Torres, Sofia Almeida) — Utilizadores primários da frente de loja. Interesses: sistema rápido que centralize o histórico do cliente de forma unívoca, criação de OS sem redundância de dados, respostas claras sobre o estado das reparações.

Mecânicos e Especialistas Técnicos (Tiago Mendes, João “Ruca” Silva e equipas regionais) — Utilizadores operacionais. Interesses: interface simplificada adaptada a ecrãs de oficina, associação imediata das peças à OS para abate automático de stock, registo fluido de tempos de mão de obra.

Clientes Finais (Proprietários de Trotinetes) — Beneficiários diretos do serviço. Interesses: total transparência nos orçamentos, celeridade na entrega dos equipamentos, comunicações atempadas sobre o estado da trotinete.

Fornecedores de Peças e Componentes — Parceiros de negócio externos. Interesses: receber pedidos de encomenda de forma estruturada e atempada, impulsionados pelos alertas automáticos de quebra de stock mínimo.

Equipa de Desenvolvimento de Software — Entidade tecnológica executante. Interesses: entregar uma solução robusta, escalável e segura dentro do prazo e orçamento acordados.

1.8 Planeamento e Cronograma do Projeto

O desenvolvimento do sistema DLMCare segue uma metodologia sequencial e incremental. O cronograma contempla as quatro etapas principais: Engenharia de Requisitos, Arquitetura e Design, Implementação e Avaliação de Qualidade, culminando na entrega final em maio de 2026.

[Diagrama de Gantt — Cronograma do Projeto DLMCare]

2. Levantamento e Análise de Requisitos

2.1 Método de Levantamento e de Análise de Requisitos Adotado

O levantamento de requisitos foi conduzido pelos analistas Francisco Soares e Rodrigo Rocha com o objetivo de identificar e estruturar as necessidades funcionais e informacionais da cadeia de oficinas DLMCare. O intuito principal foi apoiar o desenvolvimento de um sistema cloud centralizado para a gestão de clientes, frota de trotinetes, ordens de serviço e controlo de inventário em ambiente multi-loja. Para garantir que a solução a desenvolver estivesse perfeitamente alinhada com as reais necessidades operacionais e de expansão da DLMCare, foi adotada uma abordagem mista, utilizando várias técnicas de engenharia de requisitos:

- **Entrevistas:** Numa primeira fase, foram conduzidas entrevistas semiestruturadas com os principais stakeholders da empresa. Isto incluiu o CEO e fundador (David Machado), os gerentes de loja, os mecânicos e os clientes finais, permitindo mapear o fluxo de trabalho atual e os estrangulamentos na comunicação entre filiais.
- **Questionários:** Para obter dados quantitativos e uma visão mais abrangente do ecossistema do negócio, foram aplicados questionários em duas frentes: aos funcionários, com o objetivo de quantificar o tempo perdido em tarefas manuais e na procura de peças; e aos clientes finais, visando compreender as suas principais dores, expectativas quanto aos tempos de reparação e a necessidade de transparência no estado das suas trotinetes.

2.1.1 Realização de Entrevistas

Stakeholder 1: David Lopes Machado (CEO / Product Owner)

Tom: Visão estratégica, focado na rentabilidade, expansão e eliminação de desperdícios operacionais.

P1: “David, como CEO, que tipo de dados precisas de ver nos relatórios financeiros e técnicos?”

R: “Preciso de um dashboard analítico que me tire da ‘gestão por palpite’. A nível técnico, quero ver o volume de trotinetes reparadas por mês, os tempos médios de reparação e os rácios de eficiência de cada mecânico. A nível financeiro, é crítico ter o apuramento exato do lucro líquido mensal, a análise de receitas brutas e as despesas com a aquisição de peças a fornecedores.”

P2: “Como queres que o sistema faça a faturação dos serviços prestados e das peças comercializadas?”

R: “Tem de ser um processo 100% automático e transparente, sem espaço para o esquecimento humano que temos tido com o papel. O sistema tem de cruzar o valor tabelado do tipo de serviço com o custo de venda exato das peças que foram abatidas do inventário. O sistema calcula o valor final e emite a faturação detalhada de serviços e artigos, garantindo que maximizamos a margem de lucro e não deixamos dinheiro na mesa.”

P3: “Quais são as principais falhas que sentes na operação atual?”

R: “Temos três cancros: (1) Caos Descentralizado — falta de histórico de clientes entre lojas; (2) Gestão de Stock Cega — encomendar controladores para Lisboa quando o Porto tem excedentes; (3) Falhas de Comunicação — apoio ao cliente dependente de grupos de WhatsApp.”

P4: “Tens alguma preocupação com o controlo interno e a segurança?”

R: “Totalmente. Em primeiro lugar, a nossa realidade operacional não é igual em todo o lado; os custos no Porto não são os mesmos de Braga, por isso preciso de conseguir parametrizar o custo dos serviços para cada loja de forma independente. Em segundo lugar, para proteger o nosso dinheiro, exijo um registo de auditoria rigoroso. O José (nosso gerente) precisa de saber exatamente quem, quando e a que horas alterou manualmente o stock de uma peça para evitar ‘desvios’ misteriosos. E claro, como vamos ter dados sensíveis de clientes e faturação na cloud, a segurança tem de ser máxima, com as passwords da equipa fortemente encriptadas.”

—

Stakeholder 2: Gestores de Loja (interlocutora: Sofia Almeida)

Tom: Organizada, focada no atendimento rápido ao cliente e exausta do caos dos grupos de WhatsApp.

P1: “Que informações exatas precisam para fazer o registo do cliente e do equipamento?”

R: “Precisamos de uma base de dados relacional que nos permita fazer um cadastro único. Para o equipamento, precisamos obrigatoriamente de associar a marca, o modelo e o número de série ao perfil do proprietário, garantindo identificação unívoca em qualquer loja da rede.”

P2: “Como é que gostariam que fosse o processo de criação de ordens de serviço?”

R: “Queremos um módulo central onde a criação da OS seja rápida e sem redundância de dados. O sistema deve impor uma terminologia padronizada — atualmente o Porto descreve uma avaria de uma maneira e Lisboa de outra.”

P3: “Como é que o sistema vos poderia facilitar a vida na atualização do estado da reparação?”

R: “Atualmente é um pesadelo: dependemos de folhas de papel que se sujam na oficina e de mensagens no grupo ‘Mecânicos DLMCare’ que demoram horas a ser respondidas. O novo sistema vai salvar-nos a vida se permitir atualizar o estado da reparação em tempo real com um clique (ex: ‘Em Avaliação’, ‘Aguardar Peças’, ‘Em Reparação’, ‘Concluído’). Assim, quando o cliente liga, a resposta está logo ali no ecrã.”

P4: “Como vê a questão da privacidade e segurança no dia a dia?”

R: “Temos de cumprir a lei do RGPD à risca para evitar problemas. No momento de registar um cliente novo no sistema, precisamos de uma forma clara (uma checkbox, por exemplo) de registar o consentimento explícito para o tratamento dos dados pessoais dele. Outra coisa prática: como partilhamos terminais e às vezes temos de ir a correr à oficina falar com o Ruca, o sistema devia terminar a sessão automaticamente ao fim de algum tempo de inatividade, tipo 60 minutos. Assim garantimos que não deixamos os dados dos clientes ou as faturas expostos no ecrã.”

—

Stakeholder 3: Mecânicos Especializados (interlocutor: João “Ruca” Silva)

Tom: Prático, direto, focado na parte técnica e sem paciência para burocracias de papel.

P1: “Como é que o sistema deve ser desenhado para facilitar o registo do diagnóstico?”

R: “Tem de ser desenhado para quem está ‘com a mão na massa’. Precisamos de uma interface simplificada, limpa e ergonómica a correr em tablets nas bancadas da oficina.”

P2: “Como preferem registar as peças utilizadas, os tempos de reparação e os estados do serviço?”

R: “De forma fluida. O ideal é, no próprio tablet da bancada, irmos picando ou selecionando as peças que estamos a usar na trotinete para que elas fiquem logo associadas à OS. O registo de tempos de mão de obra tem de ser simples (talvez um botão de start/stop na intervenção) e a mudança de estado (ex: ‘Em Reparação’ para ‘Concluído’) deve estar à distância de um toque no ecrã. Fim das folhas de obra em papel!”

P3: “De que forma querem ser avisados de que uma peça está a acabar?”

R: “Nós não queremos andar a avisar ninguém; o sistema é que tem de trabalhar por nós. Ao associarmos uma peça à reparação no tablet, o sistema deve abater esse material do inventário de forma dinâmica. Para as peças críticas (baterias, pneus, controladores, pastilhas), o sistema deve ter alertas automáticos de stock mínimo, para que o David ou os gerentes saibam que têm de encomendar (ou transferir do Porto) antes de nós ficarmos com trotinetes paradas semanas à espera de material.”

—

Stakeholder 4: Clientes Finais (interlocutor: Sr. Costa)

Tom: Exigente, procura conveniência, transparência e confiança no serviço pago.

P1: “De que forma gostava de ser notificado sobre a evolução do estado do serviço?”

R: “Gostava de receber comunicações atempadas, exatas e automáticas (por SMS ou email, por exemplo) sempre que o estado da minha trotinete muda (ex: quando está a aguardar peças ou quando está finalmente pronta a levantar). Não quero ter de ligar para a linha geral e ficar à espera que a rececionista vá tentar descobrir em que loja está a minha Ninebot amarela.”

P2: “Que nível de detalhe gostava de ter sobre os custos na sua fatura?”

R: “Exijo total transparência. Quero que o orçamento e a fatura final mostrem claramente a separação entre o que estou a pagar em peças e o que estou a pagar em mão de obra especializada. Saber que não há peças esquecidas ou custos ‘surpresa’ dá-me segurança para voltar.”

P3: “Seria útil ter acesso a um portal ou área de cliente para ver o histórico das intervenções?”

R: “Sem dúvida. Se eu consertei a trotinete em Lisboa o ano passado e agora me mudei para Braga, quero chegar à loja da Avenida da Liberdade e saber que eles reconhecem a minha trotinete, sabem exatamente o que foi trocado na última revisão e respeitam as garantias. Isso é a verdadeira experiência de excelência.”

2.1.2 Survey Quantitativo (Questionários)

Foram realizados dois questionários estruturados para validar a aceitação de novas funcionalidades e medir a dor operacional das equipas.

A. Questionário para Clientes Finais (Foco: Experiência e Transparência)

1. Numa escala de 1 a 5, quão importante seria receber um email automático assim que a sua trotinete estiver pronta para levantamento?
2. O facto de o seu histórico ser reconhecido em qualquer loja DLMCare aumentaria a sua fidelização à marca? (Escala 1-5)
3. Qual o principal motivo de insatisfação atual? () Tempo de espera () Falta de comunicação () Erros no orçamento

B. Questionário para Staff (Mecânicos e Rececionistas)

1. Com que frequência o uso de grupos de WhatsApp causa confusão sobre o estado real de uma reparação? (Escala 1-5)
2. Quão confortável se sentiria a utilizar um tablet em vez de papel para registar peças e tempos de mão de obra? (Escala 1-5)
3. Quanto tempo gasta, em média, por dia a tentar decifrar folhas de obra manuscritas ou a procurar peças no Excel? () <30 min () 30–60 min () >60 min
4. A interface do sistema deve permitir operações com apenas uma mão/toques rápidos? (Escala 1-5)
5. Qual a maior barreira para a eficiência hoje? () Falta de peças em stock () Excesso de burocracia em papel () Falha na comunicação entre lojas

Para validar os requisitos e priorizar o MVP, foram simuladas respostas de 30 clientes e 12 elementos do staff operacional.

Relativamente aos clientes finais, a análise das 30 respostas simuladas revela uma forte necessidade de modernização e transparência. A esmagadora maioria, cerca de 90%, atribuiu a importância máxima à receção de notificações automáticas assim que a trotinete se encontra pronta para levantamento. Adicionalmente, 85% dos clientes confirmaram que o reconhecimento do seu histórico em qualquer loja aumentaria significativamente a sua fidelização à marca. Por fim, 70% apontaram a falta de comunicação e os erros nos orçamentos como as suas maiores queixas — validando a

necessidade de faturação baseada em valores tabelados por tipo de serviço (serviço tabelado + peças vendidas ao preço de venda).

Relativamente ao staff, 75% indicam que o uso do WhatsApp gera confusão na comunicação, evidenciando a urgência de centralizar as OS. A transição digital é facilitada pelo facto de 83% se sentirem confortáveis a usar tablets (validando RNF02, RNF03 e RNF04). Em contrapartida, 67% perdem mais de 30 minutos diários a lidar com papel e Excel.

Prompt

Com base nos questionários já definidos para clientes e staff da DLMCare, simula resultados quantitativos plausíveis e escreve uma interpretação curta que permita justificar a prioridade das funcionalidades de notificações, interface em tablet e eliminação de papel/Excel.

Análise Crítica

A resposta foi integrada porque a secção já tinha as perguntas do survey, mas não apresentava qualquer leitura quantitativa. A equipa validou os valores como dados simulados e não como resultados reais, usando-os apenas para reforçar a coerência entre a eliciação e os requisitos priorizados.

Modelo: ChatGPT

2.1.3 Relatório de Análise de Artefatos Técnicos

Foi efetuada uma análise às ferramentas de suporte atuais — folhas de obra físicas e ficheiros Excel de inventário.

Observações da Análise:

- **Folhas de Obra (Papel):** A exposição a óleos e químicos de oficina torna os registos frequentemente ilegíveis.
- **Excel de Stock:** O ficheiro apresenta múltiplas versões e falta de integridade transacional, resultando em dados obsoletos sobre peças críticas.

Dados Críticos Frequentemente em Falta/Ilegíveis:

1. **Timestamp de Intervenção (Início/Fim):** Os mecânicos raramente registam a hora exata, levando a estimativas vagas de produtividade.
2. **Referência Exata da Peça:** A descrição manual (ex: “controlador”) é ambígua, dificultando o abate correto no catálogo de stock centralizado.
3. **Preço de Venda da Peça:** O sistema deve guardar o preço de venda da peça no momento da aplicação, para que faturas antigas não sejam alteradas por mudanças de preço futuras.

Prompt

Contexto: Sou um Engenheiro de Requisitos a trabalhar no sistema DLMCare. Atualmente, temos apenas dados de reuniões qualitativas. Preciso de expandir a eliciação utilizando dois novos métodos: Survey Quantitativo e Análise de Artefatos. Gera: (1) questionários de 5 perguntas para Clientes e para Staff; (2) relatório de análise técnica às folhas de obra e Excel; (3) identificação de 3 campos de dados críticos frequentemente ilegíveis ou em falta. Apresenta os resultados de forma estruturada, pronta a inserir no relatório.

Análise Crítica

A resposta foi bastante completa. Fizemos pequenas alterações em relação aos índices, e à parte final, onde a LLM enunciou requisitos levantados nestas interações — acreditamos ficar melhor junto do resto dos requisitos.

Modelo: Gemini-3

2.2 User Stories

Épico 1: Gestão de Clientes e Frente de Loja

- **US01 — Registo Centralizado:** Como Gestor de Loja, quero registar o perfil de um cliente e a sua trotinete (marca, modelo e número de série), para que o seu histórico fique imediatamente acessível em qualquer loja da rede DLMCare. Critérios de aceitação: o sistema não deve permitir o registo de duas trotinetes com o mesmo número de série; deve ser possível pesquisar um cliente existente pelo NIF ou número de telemóvel antes de criar um novo registo, evitando duplicações.
- **US02 — Emissão de Fatura Automática:** Como Gestor de Loja, quero que o sistema calcule o valor final da OS (somando peças e mão de obra) com um clique, para que possa emitir uma fatura transparente e sem risco de esquecer a cobrança de material. Critérios de aceitação: a fatura gerada deve discriminar claramente as linhas de “Peças Aplicadas” e “Tipo de Serviço”.

Épico 2: Gestão de Oficina e Ordens de Serviço

- **US03 — Atualização de Estado:** Como Mecânico, quero alterar o estado da reparação através do tablet da oficina, para que a receção saiba o ponto de situação em tempo real sem me ter de ir perguntar. Critérios de aceitação: interface responsiva e adaptada a ecrãs touch; alteração de estado refletida no sistema central em menos de 2 segundos.
- **US04 — Registo de Tempos:** Como Mecânico, quero usar um botão de start/stop na OS para registar o tempo que demorei na intervenção, para que as minhas horas produtivas sejam contabilizadas. Critérios de aceitação: o sistema deve acumular o tempo se a intervenção for pausada e retomada mais tarde.

Épico 3: Gestão de Inventário

- **US05 — Abate Automático de Stock:** Como Mecânico, quero selecionar e associar as peças que estou a usar diretamente à OS, para que estas sejam abatidas automaticamente do inventário da minha loja. Critérios de aceitação: o sistema deve impedir a seleção de uma peça se o stock virtual dessa loja for zero, emitindo um aviso de rutura.
- **US06 — Alertas de Stock Mínimo:** Como CEO/Gestor, quero receber um alerta automático quando o stock de uma peça crítica atingir o limite mínimo, para que possa encomendar ou transferir material antes de imobilizar trotinetes. Critérios de aceitação: limite mínimo parametrizável por loja e por tipo de peça; alerta visível no Dashboard principal.

Épico 4: Relatórios e Apoio à Decisão

- **US07 — Dashboard Analítico:** Como CEO, quero aceder a um dashboard com as métricas de lucro líquido, tempos médios de reparação e eficiência por mecânico, para que possa comparar a rentabilidade real entre as lojas. Critérios de aceitação: filtros por intervalo de datas e por loja; apenas utilizadores com perfil de “Administrador” têm acesso.

Épico 5: Experiência do Cliente

- **US08 — Notificações de Estado:** Como Cliente, quero receber uma notificação (email) sempre que a minha trotinete mudar de estado, para que não tenha de ligar para a linha geral a perguntar

pelo ponto de situação. Critérios de aceitação: envio acionado automaticamente no momento em que o mecânico muda o estado no tablet.

2.3 Especificação dos Requisitos Levantados

2.3.1 Requisitos Funcionais (RF)

Nr.	Data e Hora	Descrição	Área	Fonte	Ana- lista
RF01	09/03/2026 14:30	O sistema deve guardar os dados dos clientes e das suas trotinetes.	Gestão de Cli- entes	Sofia Al- meida	F. Soares
RF02	09/03/2026 14:35	O sistema tem de registar a marca, modelo e número de série de cada trolinete.	Gestão de Cli- entes	Sofia Al- meida	F. Soares
RF03	09/03/2026 14:40	O sistema deve permitir a criação de Ordens de Serviço (OS).	Gestão de OS	Sofia Al- meida	F. Soares
RF04	09/03/2026 14:45	As OS têm de ter todas a mesma linguagem para não haver confusões entre as lojas.	Gestão de OS	Sofia Al- meida	F. Soares
RF05	09/03/2026 14:50	O sistema tem de mostrar se a trolinete está em avaliação, pendente, em reparação ou concluída.	Gestão de OS	Sofia Al- meida	F. Soares
RF06	09/03/2026 10:15	O David precisa de um dashboard analítico para ver o lucro.	Relató- rios	D. L. Ma- chado	R. Rocha
RF07	09/03/2026 10:25	O sistema tem de gerar relatórios com tempos registados, receitas de serviços e as peças vendidas.	Relató- rios	D. L. Ma- chado	R. Rocha
RF08	09/03/2026 10:35	O inventário tem de ser centralizado.	Gestão de Stock	D. L. Ma- chado	R. Rocha
RF09	10/03/2026 09:45	O sistema tem de tirar as peças do stock automaticamente quando os mecânicos as usam.	Gestão de Stock	J. “Ruca” Silva	F. Soares
RF10	10/03/2026 09:55	O sistema tem de avisar o David quando há poucas peças críticas.	Gestão de Stock	J. “Ruca” Silva	R. Rocha
RF11	10/03/2026 10:05	O sistema tem de avisar quando as pastilhas e pneus estão a acabar.	Gestão de Stock	J. “Ruca” Silva	F. Soares
RF12	09/03/2026 10:45	O sistema tem de calcular o preço da reparação sozinho sem errar.	Fatura- ção	D. L. Ma- chado	F. Soares
RF13	10/03/2026 16:15	A fatura deve mostrar o serviço prestado separado das peças aplicadas pelo preço de venda.	Fatura- ção	Sr. Costa	R. Rocha
RF14	10/03/2026 16:25	O sistema deve mandar um email ao cliente quando o estado da OS mudar.	Comu- nicação	Sr. Costa	R. Rocha

Nr.	Data e Hora	Descrição	Área	Fonte	Ana- lista
RF15	11/03/2026 11:45	O sistema deve registrar automaticamente quem alterou o estado de uma OS e em que momento.	Gestão de OS	Questio- nário	R. Rocha
RF16	11/03/2026 12:30	O sistema deve suportar a aplicação de descontos comerciais (percentuais ou fixos) em campanhas de fidelização de clientes.	Fatura- ção	Artefac- tos	F. Soares
RF17	11/03/2026 14:30	O sistema deve enviar um alerta interno ao Gerente de Loja quando uma OS ultrapassa o “Tempo Médio de Reparação” definido para aquele tipo de avaria.	Notifica- ções	José Bar- ros	R. Rocha
RF18	09/03/2026 14:55	O sistema deve obrigar à recolha de consentimento explícito (checkbox) para o tratamento de dados pessoais no momento do registo de um novo cliente.	Segu- rança	Sofia Al- meida	F. Soares
RF19	09/03/2026 10:55	O sistema deve manter um log (registo histórico) de todas as alterações manuais feitas ao inventário, identificando o utilizador, a data, a hora e a quantidade alterada.	Audito- ria	D. L. Ma- chado	R. Rocha

2.3.2 Requisitos Não Funcionais (RNF)

Nr.	Data e Hora	Descrição	Área	Fonte	Ana- lista
RNF01	10/03/2026 10:15	O sistema tem de ser muito rápido a abrir as Ordens de Serviço.	Desempe- nho	J. “Ruca” Silva	F. Soares
RNF02	10/03/2026 10:20	A interface para a oficina tem de ser limpa e fácil de usar.	Usabili- dade	J. “Ruca” Silva	R. Rocha
RNF03	10/03/2026 10:25	Os mecânicos não podem perder muito tempo no dispositivo.	Usabili- dade	J. “Ruca” Silva	R. Rocha
RNF04	10/03/2026 10:30	O sistema deve funcionar em ta- blets na oficina.	Plataforma	J. “Ruca” Silva	R. Rocha
RNF05	09/03/2026 15:05	O sistema não pode perder dados de clientes ou do stock.	Segu- rança / Fiabilidade	Sofia Al- meida	F. Soares
RNF06	09/03/2026 11:15	O sistema deve usar a cloud para sincronizar tudo em tempo real.	Arquite- tura	D. L. Ma- chado	F. Soares
RNF07	09/03/2026 11:20	O sistema deve encriptar as pas- swords dos utilizadores na base de dados utilizando algoritmos de hashing robustos.	Segurança	D. L. Ma- chado	F. Soares

Nr.	Data e Hora	Descrição	Área	Fonte	Ana- lista
RNF08	09/03/2026 15:10	O sistema deve terminar automaticamente a sessão de um utilizador (timeout) após 60 minutos de inatividade nos terminais de loja ou oficina.	Sessão	Sofia Almeida	R. Rocha

2.4 Análise, Validação e Refinamento de Requisitos

Durante a fase de eliciação, a recolha de informação junto dos stakeholders gerou um conjunto inicial de requisitos formulados em linguagem natural. Como é comum nesta fase, muitos destes requisitos apresentavam problemas de ambiguidade, redundância, subjetividade ou falta de testabilidade.

De forma a cumprir as boas práticas de Engenharia de Software e a preparar a Especificação de Requisitos para a equipa de desenvolvimento, procedeu-se ao refinamento rigoroso da lista inicial.

Resolução de Subjetividade e Falta de Testabilidade

Requisitos que contêm adjetivos como “rápido”, “fácil” ou “limpo” são impossíveis de testar objetivamente. Estes foram reescritos para incluir métricas quantificáveis.

Requisito Original (Em Bruto)	Problema Identificado	Requisito Refinado (Versão Final)
RNF01: O sistema tem de ser muito rápido a abrir as Ordens de Serviço.	“Muito rápido” é subjetivo. Como é que a equipa de testes valida isto?	RNF01.1: O sistema deve carregar e apresentar os detalhes de uma OS em menos de 2 segundos, para 95% das solicitações em condições normais de rede.
RNF02/RNF03: A interface para a oficina tem de ser limpa e fácil de usar. Os mecânicos não podem perder muito tempo.	“Limpa” e “fácil” não são mensuráveis. “Muito tempo” é vago.	RNF02.1: A interface da oficina deve permitir que a alteração de estado de uma OS e a associação de uma peça sejam realizadas com um máximo de 3 interações (cliques/toques no ecrã).
RNF05: O sistema não pode perder dados de clientes ou do stock.	Falta de métrica técnica. “Não perder” não é um requisito testável por si só.	RNF05.1: O sistema deve garantir a persistência dos dados através de backups incrementais diários com redundância geográfica, garantindo um RPO (Recovery Point Objective) de 24 horas.
RNF06: O sistema deve usar a cloud para sincronizar tudo em tempo real.	“Tempo real” e “tudo” são termos imprecisos.	RNF06.1: As atualizações de inventário e estados de OS efetuadas numa filial devem ser replicadas na BD central e visíveis para as restantes lojas num máximo de 5 minutos.

Requisito Original (Em Bruto)	Problema Identificado	Requisito Refinado (Versão Final)
RF10/RF11: O sistema tem de avisar o David quando as peças estão a acabar.	Uso de nome próprio e termo vago (“acabar”). Não define o canal de aviso.	RF10.1: O sistema deve gerar um alerta visual de alta prioridade no dashboard de gestão sempre que o stock de um artigo crítico atinja o limite mínimo configurado por loja.
RF12: O sistema tem de calcular o preço da reparação sozinho sem errar.	Informalidade e falta de definição da regra de negócio para o cálculo.	RF12.1: Valor Tabelado do Serviço + Somatório do Preço de Venda das Peças Aplicadas (o preço de compra das peças é interno, e o preço usado na fatura fica congelado).

Correção de Linguagem Informal e Foco na Solução

Requisitos focados em pessoas específicas ou formulados de forma coloquial foram ajustados para refletir o comportamento esperado do sistema.

Requisito Original (Em Bruto)	Problema Identificado	Requisito Refinado (Versão Final)
RF06: O David precisa de um dashboard analítico para ver o lucro.	O requisito não deve focar-se numa pessoa específica, mas sim num papel de utilizador.	RF06.1: O sistema deve disponibilizar um Dashboard de Gestão, acessível a utilizadores com perfil de “Administrador”, contendo as métricas de lucro líquido, receitas agregadas e volume de intervenções por loja.
RF04: As OS têm de ter todas a mesma linguagem para não haver confusões.	Não define como o sistema resolve o problema da linguagem.	RF04.1: O sistema deve obrigar à utilização de um catálogo padronizado (menus dropdown ou tags predefinidas) para a descrição de avarias e intervenções nas OS.

Prompt

Dá-me uma lista dos requisitos funcionais e não funcionais que parecem estar um bocado ambíguos, e que não se encontram refinados na secção 2.4. Diz-me também o que devo colocar, de acordo com a estrutura que já lá está.

Análise Crítica

Foi necessário colocar esta prompt, visto que foram adicionados novos requisitos. A resposta continha alguns pontos repetidos e não importantes, que foram filtrados.

Modelo: Gemini-3

2.5 Modelação de Casos de Uso

Com base no levantamento e na análise detalhada dos requisitos recolhidos, procedeu-se à modelação funcional do sistema DLMCare. Os Casos de Uso apresentados nesta secção traduzem as necessidades operacionais e de negócio em funcionalidades concretas de software.

UC01: Registar Cliente e Trotinete

- Ator principal: Gestores de Loja e Apoio ao Cliente (ex: Inês, Miguel, Sofia)
- Pré-condições: utilizador autenticado no sistema com perfil de Receção
- Pós-condição: o cliente e a trotinete ficam registados de forma centralizada, acessíveis na rede DLMCare

Fluxo Normal:

1. O gestor pesquisa o cliente pelo NIF ou contacto móvel
2. O sistema indica que o cliente não existe
3. O gestor obtém o consentimento do cliente
4. O gestor introduz os dados pessoais do cliente
5. O gestor introduz os dados da trotinete (marca, modelo, número de série)
6. O sistema valida o número de série (garante que é único)
7. O sistema guarda o registo e associa a trotinete ao cliente

Fluxo de Exceção 1 — Número de série inválido (Passo 6): o sistema emite um alerta de duplicação e cancela o registo.

Fluxo de Exceção 2 — Cliente não consente (Passo 3): o sistema cancela o registo.

Fluxo Alternativo 1 — Cliente já existe (Passo 2): vai para o passo 4.

—

UC02: Registar Diagnóstico e Abater Peças

- Ator principal: Mecânico / Especialista Técnico (ex: Tiago, Ruca)
- Pré-condições: OS criada; mecânico autenticado num tablet da oficina
- Pós-condição: a OS fica com peças e tempos associados; o inventário é atualizado em tempo real

Fluxo Normal:

1. O mecânico abre a OS associada à trotinete
2. O mecânico inicia a intervenção, ativando a contagem de tempo
3. O mecânico seleciona no catálogo as peças que vai aplicar
4. O sistema verifica a disponibilidade de stock local e associa as peças à OS
5. O sistema guarda a quantidade e o preço de venda unitário no momento
6. O sistema abate automaticamente as peças do inventário da loja correspondente
7. O mecânico conclui a intervenção e o sistema regista o tempo final

Fluxo de Exceção 1 — Peça indisponível (Passo 4): o sistema bloqueia a seleção e emite um alerta visual. A eventual encomenda externa ao fornecedor é tratada fora do sistema, de acordo com a restrição definida na secção 1.5.1.

—

UC03: Alterar Estado da Ordem de Serviço

- Ator principal: Mecânico / Especialista Técnico
- Pré-condições: OS em curso; mecânico autenticado no tablet da oficina
- Pós-condição: o estado da reparação é atualizado no sistema central e o cliente recebe uma notificação automática

Fluxo Normal:

1. O mecânico acede à OS da trotinete no tablet
2. O mecânico seleciona a opção de alterar estado
3. O mecânico escolhe o novo estado num catálogo (“Por iniciar”, “Aguardar peças”, “Em reparação”, “Concluído”)
4. O sistema guarda o novo estado e atualiza a informação em tempo real na receção
5. O sistema envia um email ao cliente a informar do novo estado

—

UC04: Emitir Fatura

- Ator principal: Gestor de Loja
- Pré-condições: OS encontra-se em estado “Concluído”; gestor autenticado no terminal da receção
- Pós-condição: fatura gerada pronta para ser entregue ao cliente

Fluxo Normal:

1. O gestor pesquisa e abre a OS concluída do cliente
2. O gestor clica na opção para gerar faturação
3. O sistema cruza os dados do valor do serviço registado com o custo das peças vendidas
4. O sistema apresenta um resumo do valor final, separando claramente os custos de peças dos custos de mão de obra
5. O gestor verifica os valores
6. O sistema emite a fatura certificada e gera o PDF

Fluxo Alternativo 1 — Valores incoerentes (Passo 5): o gestor corrige os valores e volta ao passo 6.

—

UC05: Aceder ao Dashboard Analítico

- Ator principal: CEO (David Lopes Machado)
- Pré-condições: utilizador autenticado com perfil de “Administrador”
- Pós-condições: as métricas financeiras e operacionais são exibidas

Fluxo Normal:

1. O CEO acede ao menu “Dashboard” do sistema
2. O sistema processa os dados centrais e apresenta a visão global da rede de lojas
3. O CEO aplica filtros por intervalo de datas e por loja específica (ex: comparar Porto e Braga)
4. O sistema atualiza os gráficos exibindo o volume de trotinetes reparadas, rácios de eficiência por mecânico, lucro líquido exato, etc.

Fluxo Alternativo 1 — Aplicação de filtros: o CEO seleciona filtros por loja, intervalo temporal, mecânico ou tipo de avaria; o sistema recalcula e atualiza os gráficos; o utilizador pode limpar os filtros para regressar à visão global.

—

UC06: Consultar Histórico do Cliente

- Ator principal: Rececionista
- Pré-condições: cliente deve estar registado no sistema; rececionista autenticado
- Pós-condição: rececionista visualiza as intervenções passadas do cliente e descarrega as faturas relevantes

Fluxo Normal:

1. O rececionista pesquisa no sistema o NIF do cliente

2. O sistema lista o parque de trotinetes em nome do cliente
3. O gestor seleciona uma trotinete
4. O sistema exibe o histórico de intervenções, garantias e reparações efetuadas nessa trotinete
5. O rececionista clica numa entrada para efetuar o download da fatura

—

UC07: Reabastecer Stock

- Ator principal: Gerente de Loja
- Pré-condições: o gerente encontra-se autenticado no sistema e as peças encomendadas externamente já foram entregues fisicamente na loja
- Pós-condição: as quantidades das peças selecionadas são incrementadas de forma persistente na base de dados do inventário da loja

Fluxo Normal:

1. O gerente de loja acede ao módulo de “Inventário” e seleciona “Registrar Entrada de Stock”
2. O sistema apresenta a interface de pesquisa e inserção de artigos
3. O gerente introduz o termo de pesquisa da peça pretendida
4. O sistema processa a pesquisa e exibe os detalhes do artigo correspondente
5. O gerente insere a quantidade de unidades recebidas
6. O gerente submete a operação clicando em “Confirmar Entrada”
7. O sistema valida os dados e atualiza o stock total da peça na base de dados
8. O sistema exibe uma mensagem de sucesso e retorna à visualização geral do inventário atualizado

Fluxo Alternativo 1 — Artigo Inexistente (Passo 4): o gerente seleciona “Adicionar Novo Artigo ao Catálogo”, preenche os dados obrigatórios e prossegue para o passo 5.

Fluxo Alternativo 2 — Inserção de quantidades inválidas (Passo 6): o sistema exibe uma mensagem de erro; o gerente corrige o valor e volta ao passo 6.

Prompt

Casos de Uso (Use Cases): Escreve a especificação textual de pelo menos 4 Casos de Uso principais do sistema DLMCare, incluindo Ator Principal, Pré-condições, Fluxo Principal, Fluxos Alternativos e Pós-condições.

Análise Crítica

Para além da prompt apresentada, foi também fornecido previamente todo o contexto do trabalho (histórico da DLMCare, stakeholders e lista de requisitos). Foram apenas realizados pequenos ajustes e correções nos fluxos alternativos e de exceção.

Modelo: Gemini-3

2.6 Diagrama de Casos de Uso

Com base nos casos de uso acima apresentados, foi desenvolvido um diagrama de casos de uso que os representava e relacionava com os seus respetivos atores.

[Diagrama de Casos de Uso — Sistema DLMCare (PlantUML)]

Prompt

Atua como um Arquiteto de Software Especialista em UML. Preciso que crie o código PlantUML para o Diagrama de Casos de Uso do sistema “DLMCare”. Atores: Gerente de Loja, Rececionista, Mecânico, CEO/Administrador (David Machado). Casos de Uso: UC01 a UC07. Liga os atores aos casos de uso de forma lógica. Usa `left to right direction` e `skinparam packageStyle rectangle`. Output esperado: apenas o código PlantUML limpo e pronto a renderizar.

Análise Crítica

O diagrama apresentado foi obtido com esta prompt. Devido às especificações da prompt, o primeiro output foi excelente, não sendo necessárias quaisquer alterações.

Modelo: Gemini

Capítulo 2 — Arquitetura e Design do Software Utilizando LLM

A conceção arquitetural do sistema DLMCare foi orientada pela necessidade imperativa de garantir a sincronização de dados em tempo real entre as três filiais (Lisboa, Porto e Braga), assegurando simultaneamente a integridade transaccional do inventário e a alta disponibilidade da plataforma. Para responder a este desafio, a arquitetura de software foi desenhada com base em princípios de desacoplamento, escalabilidade e segurança.

3. Arquitetura Global do Sistema

3.1 Visão Geral e Decisões Arquiteturais

3.1.1 Padrão Arquitetural Adotado

Para o desenvolvimento do DLMCare, adotou-se o padrão de Arquitetura em 3 Camadas (3-Tier Architecture) assente num modelo Cliente-Servidor.

A escolha deste padrão justifica-se pela clara separação de responsabilidades (Separation of Concerns) que oferece. Ao isolar a interface de utilizador, a lógica de processamento e o armazenamento de dados em camadas independentes, o sistema ganha uma resiliência significativa. Este desacoplamento permite que a aplicação cliente seja atualizada sem impacto nas regras de negócio subjacentes. Adicionalmente, este padrão é nativamente compatível com ambientes cloud, permitindo escalar os recursos do servidor central de forma elástica, garantindo a alta disponibilidade exigida para o funcionamento ininterrupto das três oficinas.

3.1.2 Camadas Lógicas do Sistema

A arquitetura do sistema divide-se logicamente em três camadas fundamentais, cada uma com responsabilidades bem delimitadas e interfaces de comunicação claramente definidas.

A **Camada de Apresentação (Frontend)** atua como o ponto de interação exclusivo com os utilizadores, nomeadamente o Administrador, os Gerentes, os Rececionistas e os Mecânicos. Será desenvolvida como uma aplicação web com design estritamente responsivo, uma característica crítica dado que a interface tem de se ajustar dinamicamente aos desktops de ecrã largo utilizados no backoffice e na receção, e aos tablets utilizados pelos mecânicos nas bancadas de trabalho. A sua principal função é recolher os inputs dos utilizadores, enviar pedidos estruturados para o servidor e renderizar os dados recebidos de forma clara e intuitiva.

A **Camada de Lógica de Negócio (Backend/API)** constitui o núcleo de processamento do sistema DLMCare. Exposta através de uma API, esta camada intermédia centraliza todas as regras e processos do negócio, sendo responsável por receber os pedidos da camada de apresentação, validar permissões de acesso e executar operações complexas. É aqui que residem os algoritmos de cálculo automático de faturação, os mecanismos de alerta de stock mínimo e a máquina de estados que rege as transições das Ordens de Serviço (OS). A centralização da lógica no servidor garante que todas as filiais operam sob as mesmas regras, eliminando discrepâncias operacionais entre instalações.

A **Camada de Dados (Database)** será materializada através de um SGBDR centralizado e alojado na cloud. A opção por um modelo relacional é inegociável, dada a obrigatoriedade de garantir

as propriedades ACID. Este rigor transaccional é fundamental para o controlo de concorrência no inventário de peças: caso dois rececionistas tentem abater a mesma peça do stock central em simultâneo, o sistema processará os pedidos de forma sequencial e isolada, prevenindo duplicações.

[Diagrama de Arquitetura — 3 Camadas do Sistema DLMCare]

Prompt

Atua como um Arquiteto de Software Sénior. Estamos a iniciar a “Etapa 2 — Arquitetura e Design” do projeto DLMCare (sistema cloud para gestão de 3 oficinas de trotinetes). Contexto: o sistema tem de sincronizar dados em tempo real entre Lisboa, Porto e Braga; acesso via desktops e tablets; controlo rigoroso de concorrência no inventário (ACID); alta disponibilidade na cloud. Missão: escreve o texto deste subcapítulo justificando as escolhas arquiteturais. O tom deve ser académico, técnico e formal.

Análise Crítica

Diferente de uma arquitetura monolítica tradicional, que apresentaria riscos de contenção na base de dados centralizada e dificuldades de sincronização geográfica, a solução proposta oferece o equilíbrio ideal. Garante-se o rigor matemático no stock de peças e a fluidez necessária para a operação em tempo real.

Modelo: Gemini

3.2 Stack Tecnológico

A seleção do stack tecnológico para o sistema DLMCare foi conduzida segundo critérios rigorosos de engenharia de software, tendo como objetivo principal a mitigação de riscos de implementação e o alinhamento com o padrão arquitetural de três camadas previamente definido. Privilegiou-se a adoção de tecnologias de código aberto, amplamente consolidadas na indústria e suportadas por comunidades ativas, em detrimento de soluções proprietárias ou emergentes com menor maturidade comprovada. Para cada camada lógica do sistema, foram consideradas e avaliadas alternativas, e a escolha final recaiu sobre o conjunto de ferramentas que melhor equilibra robustez técnica, velocidade de desenvolvimento e sustentabilidade a longo prazo.

3.2.1 Backend (Lógica de Negócio e API): Python e FastAPI

Para a implementação da camada de lógica de negócio e exposição da Interface de Programação de Aplicações (API), a equipa avaliou diferentes ecossistemas tecnológicos, considerando a performance, a rapidez de desenvolvimento e a robustez necessária para um sistema multi-loja.

A tabela seguinte apresenta a comparação entre as principais tecnologias consideradas para esta camada:

Tecnologia	Vantagens	Desvantagens
FastAPI (Python)	Alta performance (assíncrono), documentação automática (OpenAPI) e tipagem de dados rigorosa.	Ecossistema de plugins mais recente comparado com frameworks mais antigos.

Tecnologia	Vantagens	Desvantagens
Node.js (Express)	Elevada escalabilidade em operações de I/O e permite o uso de JavaScript em todo o stack.	A gestão de concorrência em tarefas pesadas de CPU pode ser mais complexa.
Java (Spring Boot)	Extremamente robusto, seguro e com vasta adoção em sistemas corporativos críticos.	Curva de aprendizagem elevada e desenvolvimento mais verboso, o que atrasa a entrega.

Optámos pela combinação de Python com a framework FastAPI. A escolha do Python fundamenta-se na sua sintaxe concisa e elevada legibilidade — fatores que reduzem a complexidade cognitiva do código e facilitam a revisão colaborativa, permitindo uma iteração de desenvolvimento mais célere, o que é crítico num projeto com janela temporal restrita. A seleção específica do FastAPI, em detrimento de alternativas como o Django REST Framework (DRF), justifica-se pela sua arquitetura assíncrona nativa (baseada em ASGI e asyncio), vital para processar eficientemente pedidos concorrentes provenientes das três oficinas em simultâneo. Adicionalmente, o FastAPI integra nativamente a validação automática de dados (via Pydantic) e a geração de documentação interativa (OpenAPI/Swagger), assegurando um contrato de interface explícito e verificável entre o frontend e o backend desde o início do desenvolvimento, mitigando riscos de integração.

3.2.2 Base de Dados (Persistência): MySQL

A camada de persistência de dados é o alicerce que garante a consistência da informação entre as três filiais da DLMCare. Para a sua implementação, a equipa avaliou diferentes modelos e sistemas de gestão de bases de dados (SGBD).

A tabela seguinte apresenta a análise comparativa entre as tecnologias consideradas:

Tecnologia	Vantagens	Desvantagens
MySQL	Excelente performance em operações de leitura, suporte robusto a transações ACID e grande maturidade no mercado.	Menor flexibilidade para lidar com esquemas de dados não estruturados comparativamente ao NoSQL.
PostgreSQL	Suporte avançado para tipos de dados complexos e conformidade estrita com normas SQL.	Exige maior consumo de recursos do servidor e uma configuração inicial mais complexa.
MongoDB	Modelo de documentos flexível (NoSQL), ideal para prototipagem rápida e dados sem esquema fixo.	Dificuldade em garantir integridade referencial complexa de forma nativa e declarativa.

A equipa optou pelo MySQL, um Sistema de Gestão de Bases de Dados Relacionais (SGBDR) de código aberto. A preferência por um modelo relacional — em detrimento de alternativas NoSQL como o MongoDB — decorre da natureza estruturada e interdependente dos dados (clientes, veículos, ordens de serviço, stock e faturação). Um modelo NoSQL introduziria complexidade desnecessária na gestão da integridade referencial, que no MySQL é garantida nativamente através de chaves estrangeiras. A escolha justifica-se, sobretudo, pela sua conformidade estrita com as propriedades ACID, requisito não-negociável para a operação multi-instalação da DLMCare. O mecanismo de row-level locking do MySQL garante que as transações sobre stock são orquestradas de forma isolada, prevenindo anomalias que comprometeriam a faturação. Finalmente, a maturidade comprovada do MySQL reduz o risco operacional e facilita a integração com o backend em Python através de bibliotecas como SQLAlchemy, assegurando a sustentabilidade do projeto a longo prazo.

3.2.3 Frontend (Interface do Utilizador): Vue.js com HTML e CSS

A camada de apresentação é o ponto de interação fundamental entre os utilizadores e o sistema DLMCare. Para garantir uma interface fluida em múltiplos dispositivos, a equipa analisou as três principais frameworks de mercado:

Tecnologia	Vantagens	Desvantagens
Vue.js	Curva de aprendizagem suave, extremamente leve e excelente sistema de reatividade bidirecional.	Ecossistema de plugins e mercado de trabalho ligeiramente menor que o do React.
React	Ecossistema vasto, flexibilidade extrema e uma enorme biblioteca de componentes de terceiros.	Exige mais decisões de arquitetura e configuração manual (gestão de estado, routing).
Angular	Solução completa “out-of-the-box” com padrões rígidos de desenvolvimento.	Curva de aprendizagem acentuada e complexidade elevada (TypeScript rigoroso e injeção de dependências).

A camada de apresentação será desenvolvida com recurso à framework progressiva de JavaScript Vue.js, assente nos standards web de HTML e CSS. A opção pelo Vue.js em detrimento de alternativas como React ou Angular fundamenta-se num equilíbrio entre critérios de engenharia e gestão de risco do projeto. O Vue.js permite a construção de uma Single Page Application (SPA) baseada num modelo de componentes reutilizáveis. O seu sistema de reatividade de dados bidirecional (two-way data binding) assegura a sincronização automática entre o estado da aplicação e a interface visual, eliminando a manipulação direta do DOM e resultando numa interface fluida e responsiva. Esta característica é especialmente vantajosa para o módulo de Oficina, operado em tablets por técnicos em contexto de trabalho. Do ponto de vista da gestão de risco, o Vue.js apresenta uma curva de aprendizagem comprovadamente mais suave do que o Angular e uma estrutura de projeto mais explícita e previsível do que o React. Esta previsibilidade reduz a probabilidade de decisões de implementação inconsistentes entre os membros da equipa. Complementarmente, o uso de ferramentas modernas como o Vite proporciona ciclos de compilação e hot-reload rápidos, acelerando o desenvolvimento iterativo e garantindo a entrega atempada das interfaces.

Prompt

Atua como um Arquiteto de Software Sénior e Technical Writer. A nossa decisão de equipa é utilizar Python (Backend), MySQL (Base de Dados) e Vue.js com HTML/CSS (Frontend). Missão: escreve um texto justificando rigorosamente a escolha de cada tecnologia. O tom deve ser formal, académico e focado nos benefícios de engenharia e na redução de risco do projeto.

Análise Crítica

No primeiro output o que menos gostámos foi o uso de bullet points, pelo que pedimos que fossem eliminados em favor de texto corrido. O texto resultante tem a densidade e continuidade argumentativas esperadas num relatório universitário.

Modelo: Gemini e Claude

4. Modelação de Domínio

A transição do modelo de gestão rudimentar da DLMCare — assente em folhas de obra físicas e ficheiros de Excel descentralizados — para um sistema de informação cloud exige uma representação

rigorosa da sua realidade operacional. Esta secção apresenta a Modelação de Domínio e Dados, que atua como o alicerce fundamental para a integridade, persistência e manipulação da informação em toda a rede de oficinas.

Para resolver o caos logístico e as falhas de comunicação entre as filiais de Lisboa, Porto e Braga, mapearam-se as entidades do negócio (clientes, trotinetes, peças e lojas) e os respetivos processos transacionais (ordens de reparação e faturação) em abstrações lógicas de software. Este desenho orientado a objetos tem como objetivo garantir uma fonte única de verdade para todo o sistema. Dessa forma, assegura-se que o inventário virtual reflete fielmente o stock físico de cada filial e que existe uma rastreabilidade absoluta entre os tempos de mão de obra, os materiais aplicados e a faturação final. O modelo aqui definido estabelece, assim, a estrutura necessária para a posterior implementação da base de dados relacional (MySQL) e da lógica de negócio associada.

4.1 Identificação e Caracterização das Classes do Sistema

Para suportar a arquitetura e garantir o cumprimento de todas as regras de negócio operacionais, estatísticas e financeiras, o sistema assenta nas seguintes classes de domínio:

- **Hierarquia de Utilizadores (Utilizador, Administrador, GerenteLoja, Rececionista, Mecanico):** Representam os diferentes atores que interagem com o sistema, herdando credenciais de acesso comuns, mas com permissões e relações específicas.
- **Cliente:** Armazena os dados pessoais dos utilizadores e gere o consentimento RGPD.
- **Trotinete:** Identifica de forma unívoca o equipamento (número de série), centralizando o histórico.
- **Loja:** Representa as filiais físicas (Lisboa, Porto, Braga).
- **OrdemServico (OS):** A entidade central que orquestra o diagnóstico e o estado atual da intervenção.
- **RegistoTempo:** Mantido estritamente para fins estatísticos e auditoria de produtividade. Regista o tempo despendido pelos mecânicos, não tendo impacto na faturação final.
- **CatalogoServico:** Define os tipos de intervenção padrão (ex: “Substituição de Pneu”, “Revisão Eletrónica”) e os seus respetivos preços tabelados.
- **OS_Servico:** Classe associativa que regista os serviços efetivamente realizados numa OS, congelando o preço tabelado no momento da intervenção.
- **Peca:** O catálogo de artigos, com separação clara entre o preço de aquisição a fornecedores e o preço de venda ao público.
- **StockLoja:** Inventário local (quantidades e limites mínimos) específico de cada filial.
- **OS_Peca:** Classe associativa que documenta as peças consumidas numa OS, congelando o seu preço de venda para a faturação.
- **Fatura:** Documento financeiro gerado com base no somatório dos serviços tabelados e no preço de venda das peças aplicadas.
- **Auditoria:** Log de segurança do sistema.

4.2 Relacionamentos e Multiplicidades (Relacionamentos UML)

A integridade do modelo de domínio do sistema DLMCare é garantida por uma rede completa de relacionamentos estruturais e transacionais. Abaixo detalham-se todas as associações e respetivas multiplicidades presentes no diagrama:

Herança e Organização Lógica (Atores e Lojas)

- **Utilizador e Especializações (Herança/Generalização):** As classes Administrador, GerenteLoja, Rececionista e Mecanico herdam (estendem) os atributos comuns de autenticação da superclasse abstrata Utilizador.

- **Loja (1) e GerenteLoja (1..*)**: Uma filial física tem de ser administrada por pelo menos um (ou vários) gerentes; um gerente está afeto a uma única loja.
- **Loja (1) e Rececionista (1..*)**: Uma filial emprega pelo menos um rececionista para a frente de loja; cada rececionista opera apenas no sistema da sua loja.
- **Loja (1) e Mecanico (1..*)**: Uma filial tem de ter pelo menos um mecânico especializado alocado às suas bancadas.

Clientes e Equipamentos (Receção)

- **Cliente (1) e Trotinete (1..*)**: Para um cliente estar registado no sistema, tem de possuir pelo menos uma trotinete associada ao seu perfil. Uma trotinete pertence única e exclusivamente a um cliente.
- **Trotinete (1) e OrdemServico (1..*)**: Uma trotinete acabada de registar fica logo associada a uma ordem de serviço, mas ao longo do tempo poderá acumular várias ordens de serviço (*). Cada OS refere-se apenas a uma trotinete específica.

Oficina, Tempos e Serviços (Intervenção)

- **Loja (1) e OrdemServico (1..*)**: Uma OS é obrigatoriamente registada e executada numa loja específica. Uma loja acumula o histórico de múltiplas OS.
- **OrdemServico (1) e RegistoTempo (0..*)**: Uma OS pode não ter ainda tempos registados (se estiver “Por Iniciar”) ou ter vários registos de tempo (se o trabalho for pausado e retomado). Cada registo de tempo pertence apenas a uma OS.
- **Mecanico (1) e OrdemServico (1..*)**: Um mecânico pode tratar de várias ordens de serviço. Cada ordem de serviço está associada a um único mecânico.
- **OrdemServico (1) e OS_Servico (0..*)**: Uma reparação pode implicar a realização de vários serviços tabelados (ex: mudar pneu e reparar travão).
- **CatalogoServico (1) e OS_Servico (0..*)**: Um serviço do catálogo (ex: “Furo”) pode ser aplicado em milhares de OS ao longo do tempo, ou em nenhuma (se for um serviço novo).

Inventário e Aplicação de Peças

- **Peca (1) e StockLoja (0..*)**: Um artigo do catálogo global pode estar fisicamente presente no inventário de várias lojas. A classe StockLoja resolve a relação muitos-para-muitos original entre Loja e Peca.
- **Loja (1) e StockLoja (0..*)**: Uma loja contém milhares de registos de quantidades locais no seu inventário virtual.
- **OrdemServico (1) e OS_Peca (0..*)**: Uma OS pode consumir zero peças (se for apenas um serviço de limpeza/afinação) ou dezenas de peças diferentes.
- **Peca (1) e OS_Peca (0..*)**: Uma referência do catálogo pode ser aplicada em múltiplas OS diferentes. A classe OS_Peca resolve a associação muitos-para-muitos, guardando a quantidade e o preço de venda no momento.

Faturação e Segurança

- **OrdemServico (1) e Fatura (0..1)**: Uma Ordem de Serviço pode ainda não estar faturada (0) ou ter, no máximo, uma única fatura final emitida (1). A fatura pertence sempre a uma única OS.
- **Rececionista (1) e Fatura (0..*)**: Um rececionista é o responsável pela emissão legal de múltiplas faturas. Cada fatura tem o registo do rececionista que a validou.
- **Utilizador (1) e Auditoria (0..*)**: Qualquer ator do sistema (Administrador, Gerente, Mecânico ou Rececionista) pode gerar múltiplos logs de segurança ao longo do tempo (ex: alterar stock manualmente, fazer login). Cada registo de auditoria está irrevogavelmente ligado a um único utilizador.

4.3 Especificação dos Atributos das Classes OrdemService

Atributo	Descrição	Domínio	Nulo	Exemplo
idOS	Identificador único sequencial de cada OS	INT	N	1045
dataAbertura	Data e hora em que a trotinete deu entrada na oficina	DATE-TIME	N	2026/04/01 10:30
dataSaida	Data e hora em que a reparação acabou	DATE-TIME	S	2026/04/12 10:00
estadoOS	Situação atual da reparação	Enum	N	Em Reparação
diagnostico	Notas clínicas descritivas	TEXT	S	Erro E10 no visor
idTrotinete	Referência à trotinete alvo da reparação (FK)	VARCHAR(100)	N	SN998877
idLoja	Referência à loja onde a OS está a decorrer (FK)	INT	N	2

Cliente

Atributo	Descrição	Domínio	Nulo	Exemplo
idCliente	Identificador único do cliente (Chave Primária)	INT	N	1
nif	Número de identificação fiscal do cliente	VARCHAR(20)	S	234567890
nome	Nome completo do cliente	VARCHAR(150)	N	Carlos Silva
telefone	Número de telemóvel	VARCHAR(20)	N	912345678
email	Endereço de email para envio de faturas e orçamentos	VARCHAR(150)	N	carlos@email.com
consentimentoRGPD	Confirmação de aceitação dos termos de privacidade (RGPD)	BOOLEAN	N	TRUE

Utilizador (e subclasses)

Classe	Atributo	Descrição	Domínio	Nulo	Exemplo
Utilizador	idUtilizador	Identificador único	INT	N	10
	nome	Nome completo	VARCHAR(150)	N	João Ruca Silva
	email	Email do utilizador	VARCHAR(150)	N	d1m@gmail.com
	passwordHash	Password encriptada	VARCHAR(255)	S	—
Administrador	nivelAcesso	Nível de privilégio global	INT	N	1
Gerente-Loja	idLoja	Referência à loja (FK)	INT	N	1
Rececionista	idLoja	Referência à loja (FK)	INT	N	2

Classe	Atributo	Descrição	Domínio	Nulo	Exemplo
Mecanico	idLoja	Referência à loja (FK)	INT	N	3
	especiali- dade	Área de especialização	VARCHAR(100)		Eletró- nica
	comissão	Percentagem de cada serviço	INT	N	10

Trotinete

Atributo	Descrição	Domínio	Nulo	Exemplo
numSerie	Número de série de fábrica do equipamento (Chave Primária)	VARCHAR(100)		SN9988776655
marca	Fabricante da trotinete	VARCHAR(50)		Xiaomi
modelo	Modelo comercial do equipamento	VARCHAR(50)		Pro 2
idCliente	Referência ao dono da trotinete (FK)	INT	N	234567890

Peca

Atributo	Descrição	Domínio	Nulo	Exemplo
referencia	Código interno ou SKU do artigo (Chave Primária)	VARCHAR(50)		BATT- -XIA-01
nome	Designação técnica e comercial da peça	VARCHAR(150)		Bate- ria 36V 10.4Ah
precoCom- pra	Valor de aquisição a fornecedores (interno)	DECI- MAL(7,2)	N	150.00
preco- Venda	Valor pelo qual a loja vende a peça nos serviços	DECI- MAL(7,2)	N	170.00

Fatura

Atributo	Descrição	Domínio	Nulo	Exemplo
idFatura	Número sequencial e oficial do documento de fatura	INT	N	2026001
dataEmis- sao	Data e hora em que a OS foi dada como concluída e cobrada	DATE- TIME	N	2026/04/02 16:00
idRececio- nista	Referência à rececionista que emitiu a fatura (FK)	INT	N	2
valorPecas	Soma do custo de todos os materiais aplicados na reparação	DECI- MAL(7,2)	N	150.00
valorServi- cos	Soma dos preços dos serviços executados	DECI- MAL(7,2)	N	45.00
valorTotal	Valor final a pagar pelo cliente	DECI- MAL(7,2)	N	195.00
idOS	Referência à OS que originou a fatura (FK)	INT	N	1045

Loja

Atributo	Descrição	Domínio	Nulo	Exemplo
idLoja	Identificador único da filial (Chave Primária)	INT	N	2
localizacao	Cidade	VARCHAR(100)	N	Porto
morada	Morada física da loja	VARCHAR(255)	N	Rua de Júlio Dinis
contacto	Número de telefone geral da receção da loja	VARCHAR(20)	N	223344556

StockLoja

Atributo	Descrição	Domínio	Nulo	Exemplo
idLoja	Identificador da loja (FK)	INT	N	1
referencia	Referência da peça (FK)	VARCHAR(50)	N	BATT-XIA-01
quantidadeAtual	Unidades em prateleira	INT	N	5
stockMinimo	Alerta de reposição	INT	N	2

OS_Peca

Atributo	Descrição	Domínio	Nulo	Exemplo
idOS	Referência à OS intervencionada (FK)	INT	N	1045
referencia	Referência do material aplicado na reparação (FK)	VARCHAR(50)	N	PNEU-8.5-01
quantidadeAplicada	Número de unidades dessa peça gastas na intervenção	INT	N	2
precoVendaNoMomento	Preço de venda no momento da compra (congelado)	DECIMAL(7,2)	N	30.00

Auditoria

Atributo	Descrição	Domínio	Nulo	Exemplo
idLog	Identificador único do registo de segurança	INT	N	1000
idUtilizador	Referência a quem executou a ação (FK)	INT	N	5
dataHora	Timestamp do evento	DATE-TIME	N	2026/04/02 16:00
acao	Descrição da ação crítica executada	TEXT	N	Alterou o estado da OS 10

CatalogoServico

Atributo	Descrição	Domínio	Nulo	Exemplo
idServico	Identificador único do tipo de serviço	INT	N	5

Atributo	Descrição	Domínio	Nulo	Exemplo
descricao	Breve descrição do serviço	VARCHAR(200)	N	Substituição de Pneu
precoTabelado	Preço do serviço	DECIMAL(7,2)	N	10.00

OS_Servico

Atributo	Descrição	Domínio	Nulo	Exemplo
idOS	Referência à OS (FK)	INT	N	100
idServico	Referência ao serviço executado (FK)	INT	N	1
precoNo-Momento	Preço do serviço na data em que foi realizado (congelado)	DECIMAL(7,2)	N	5.00

RegistoTempo

Atributo	Descrição	Domínio	Nulo	Exemplo
idRegisto	Identificador único do registo	INT	N	1
startTime	Início da intervenção	DATE-TIME	N	2026/01/02 17:15
endTime	Fim/Pausa da intervenção	DATE-TIME	S	2026/01/02 18:00
totalMinutos	Duração calculada desta sessão	DECIMAL(7,2)	N	45
idOS	Referência à OS que estava a ser trabalhada (FK)	INT	N	5

4.4 Diagrama de Classes

[Diagrama de Classes — Modelo de Domínio DLMCare (PlantUML)]

Prompt

Atua como um Engenheiro de Software. Traduz as tabelas de atributos e relacionamentos do sistema DLMCare que acabámos de definir num script PlantUML para gerar o Diagrama de Classes (Modelo de Domínio). Garante que a classe associativa OS_Peca é representada de forma a resolver a relação Muitos-para-Muitos entre OrdemServico e Peca. Oculta os ícones de visibilidade e usa um tema monocromático formal.

Análise Crítica

A utilização do LLM para gerar o código PlantUML acelerou massivamente a documentação arquitetural. A IA interpretou corretamente a necessidade de resolver a relação N:M através de uma entidade intermédia (OS_Peca), o que aproxima o nosso modelo conceptual da realidade

do modelo lógico relacional que será implementado em MySQL na Etapa 3. O resultado foi inserido sem necessidade de refatorização adicional.

Modelo: Gemini

4.5 Consolidação e Revisão do Diagrama de Classes

A análise dos requisitos funcionais e não funcionais mostrou que o modelo inicial de classes cobre o núcleo do domínio, mas deve ser refinado antes da implementação. Em particular, a classe Peca não deve acumular simultaneamente a responsabilidade de catálogo e de stock local, uma vez que a mesma referência pode existir em várias lojas com quantidades e limites mínimos diferentes. Assim, propõe-se a consolidação do modelo de domínio através das seguintes classes e respectivas responsabilidades:

A classe Utilizador representa contas internas com credenciais e perfil de acesso, servindo de base para Administrador, Gerente, Rececionista e Mecânico. O Cliente guarda dados pessoais, contactos, NIF e consentimento RGPD, estando associado a uma ou várias trotinetes. A Trotinete representa o equipamento físico identificado por número de série único. A OrdemServico, sendo a entidade central do ciclo de reparação, agrega o diagnóstico, estado, loja, cliente, trotinete, tempos e peças. O RegistoTempo regista os períodos de trabalho de um mecânico numa OS, permitindo ações de start/stop e o cálculo de mão de obra.

Relativamente aos materiais, a PecaCatalogo define a referência, descrição, categoria e preço base da peça, não contendo stock local. O StockLoja guarda a quantidade e o limite mínimo de uma peça numa loja, utilizando uma chave composta (idLoja + referenciaPeca). A OS_Peca funciona como classe associativa entre OrdemServico e PecaCatalogo, guardando a quantidade aplicada e o preço no momento da intervenção. A TransferenciaStock regista os movimentos internos de peças entre lojas, não substituindo a compra externa a fornecedores.

A nível de faturação e controlo, a Fatura consolida peças, mão de obra, descontos e o valor final, podendo integrar-se com software certificado externo. A Notificacao regista comunicações (SMS/email) enviadas ao cliente e o estado de envio. Por fim, a Auditoria regista alterações críticas no sistema, sendo obrigatória para stock, estados de Ordem de Serviço e parametrizações. Esta separação reduz o acoplamento, facilita transações sobre inventário e evita inconsistências quando uma mesma referência existe em várias filiais.

5. Modelação Dinâmica e Componentes

A transição estrutural definida na modelação de dados exige um mapeamento claro de como a informação flui e é processada em tempo real pelas diferentes partes do sistema. Esta secção foca-se na Modelação Dinâmica e de Componentes, detalhando o comportamento da aplicação DLMCare em execução. Através da decomposição lógica em módulos independentes e da ilustração das interações temporais entre os utilizadores e a plataforma, demonstra-se como ações quotidianas da oficina — como a alteração do estado de uma reparação por um mecânico ou o registo de entrada de stock por um gerente — são orquestradas pelo backend de forma consistente. Este nível de desenho arquitetural garante que os processos complexos e concorrentes das três filiais são executados sem bloqueios, assegurando a agilidade e a sincronização exigidas para a operação ininterrupta do negócio.

5.1 Diagrama de Componentes

O diagrama de componentes representa a decomposição lógica da aplicação em módulos independentes, interligados através da API do backend. Esta visão é útil para preparar a implementação incremental da Etapa 3.

[Diagrama de Componentes — Decomposição Lógica da Aplicação DLMCare]

5.2 Diagramas de Sequência

Os Diagramas de Sequência (UML) descrevem como os objetos interagem ao longo do tempo para executar uma funcionalidade. Focam-se na ordem temporal das mensagens trocadas, ilustrando o fluxo de dados e controlo entre os componentes do sistema (interface, lógica, base de dados). São cruciais para entender a lógica de execução e identificar ineficiências ou erros no processamento da informação.

[Diagrama de Sequência — UC01: Registrar Cliente e Trotinete]

[Diagrama de Sequência — UC02: Registrar Diagnóstico e Abater Peças]

[Diagrama de Sequência — UC04: Emitir Fatura]

[Diagrama de Sequência — UC06: Consultar Histórico do Cliente]

Prompt

“Atua como Arquiteto de Software e gera o código PlantUML para os diagramas de sequência dos Casos de Uso UC01, UC02, UC04 e UC06 do projeto DLMCare. Diretrizes: (1) Arquitetura 3-Tier — todos os diagramas devem ter o fluxo Ator → Frontend (Vue.js) → Backend (FastAPI) → DB (MySQL); (2) usa as classes de consolidação da secção 4.5; (3) as chamadas entre Frontend e Backend devem usar os endpoints definidos na tabela da secção 6.1; (4) mostra o abate de stock no UC02; no UC04 mostra o cruzamento de dados para gerar a fatura final. Inclui validações de erro.”

Análise Crítica

A utilização do LLM para gerar o código PlantUML acelerou massivamente a documentação arquitetural. A IA foi capaz de analisar as classes existentes e fazer diagramas complexos conforme a arquitetura já desenhada.

Modelo: Gemini-3

6. Interfaces e Comunicação

A eficácia técnica do sistema DLMCare depende intrinsecamente da facilidade com que os utilizadores — desde os clientes finais até aos mecânicos nas bancadas de trabalho — interagem com a plataforma. Esta secção, dedicada às Interfaces e Comunicação, estabelece o contrato tecnológico e visual que materializa as operações de negócio num ambiente digital, substituindo definitivamente o uso de papel e grupos de mensagens. Aqui, documenta-se a especificação preliminar da API REST, que atua como o ponto de ligação seguro entre a infraestrutura central, os terminais das lojas e as integrações externas (como os serviços de SMS e faturação certificada). Em complemento, delinea-se a estrutura das interfaces de utilizador, garantindo que o design dos ecrãs é altamente responsivo, ergonómico e adaptado ao ritmo exigente e rápido da receção e do chão de oficina.

6.1 Especificação Preliminar da API REST

A API REST será o ponto de ligação entre o frontend Vue.js e os serviços de backend. A tabela seguinte apresenta uma especificação preliminar dos endpoints necessários para implementar os casos de uso principais.

A tabela seguinte detalha a interface de comunicação do sistema, descrevendo os endpoints disponibilizados, os métodos HTTP utilizados, a sua função principal e os perfis com permissão de acesso a cada operação:

Método	Endpoint	Descrição	Permissões
POST	/auth/login	Autentica utilizador e devolve token de sessão.	Todos
GET	/clientes?query={...}	Pesquisa cliente por NIF ou contacto.	Receção, Gerente
POST	/clientes	Cria novo cliente com consentimento RGPD.	Receção, Gerente
POST	/trotinetes	Regista trotinete associada a cliente.	Receção, Gerente
GET	/clientes/{id}/historico	Lista histórico de trotinetes, OS e faturas.	Receção
POST	/ordens-servico	Cria nova Ordem de Serviço (OS).	Receção, Gerente
PATCH	/ordens-servico/{id}/estado	Atualiza estado da OS e aciona notificações.	Mecânico, Gerente
POST	/ordens-servico/{id}/tempos/iniciar	Inicia registo de tempo de intervenção.	Mecânico
POST	/ordens-servico/{id}/tempos/parar	Termina ou pausa registo de tempo.	Mecânico

Método	Endpoint	Descrição	Permissões
POST	/ordens-servico/{id}/pecas	Associa peça à OS e abate stock.	Mecânico
GET	/stock?loja={id}	Consulta stock por loja.	Gerente, Admin
POST	/stock/entradas	Regista entrada de stock entregue fisicamente.	Gerente
POST	/stock/transferencias	Regista transferência interna entre lojas.	Gerente, Admin
POST	/faturas	Gera fatura a partir de OS concluída.	Gerente
GET	/dashboard	Obtém métricas operacionais e financeiras.	Administrador
GET	/auditoria	Consulta logs de alterações críticas.	Administrador

6.2 Design de Interfaces e Wireframes Textuais

A definição das interfaces foi orientada pelos perfis de utilizador identificados na Etapa 1. O objetivo é garantir que a implementação da Etapa 3 começa com uma visão clara dos ecrãs principais e das ações esperadas.

A tabela seguinte apresenta a estrutura dos principais ecrãs do sistema, detalhando o perfil de utilizador a que se destinam, os elementos que compõem a interface e as operações críticas que cada ecrã suporta:

Ecrã	Utilizador Principal	Elementos Principais	Ações Críticas
Login	Todos	Email, password, recuperação de acesso.	Autenticar; terminar sessão após timeout.
Pesquisa/Registo de Cliente	Rececionista	Pesquisa por NIF/contacto, dados pessoais, consentimento RGPD.	Criar cliente; associar trotinete; evitar duplicados.
Criação de OS	Rececionista	Cliente, trotinete, descrição inicial, fotografias, loja.	Abrir OS; anexar fotos; definir prioridade.
Tablet Oficina	Mecânico	Lista de OS, estado, diagnóstico, peças, start/stop de tempo.	Alterar estado; associar peças; registar tempos.
Inventário	Gerente	Peças por loja, stock atual, stock mínimo, alertas.	Registar entrada; transferir stock; consultar ruturas.
Faturação	Gerente	Resumo da OS, serviços, peças, descontos, valor final.	Validar valores; gerar fatura; descarregar PDF.
Dashbo-ard	CEO/Admin	KPIs, filtros por loja/período, gráficos e tabelas.	Comparar lojas; exportar relatório.

Nos ecrãs de oficina, as ações frequentes devem estar disponíveis com botões grandes, texto curto e contraste adequado, permitindo utilização em tablets e reduzindo o número de interações necessárias.

Prompt

Atua como Arquiteto de Software e revê a Etapa 2 do relatório DLMCare. Mantendo a arquitetura em três camadas, acrescenta os artefactos que faltam para cumprir o enunciado: revisão do modelo de domínio, diagrama de componentes, diagramas de sequência, especificação preliminar da API REST, design de interfaces/wireframes textuais, decisões arquiteturais e critérios de conclusão da Etapa 2.

Análise Crítica

A resposta foi aceite parcialmente e adaptada. A equipa manteve a arquitetura original (Vue.js, FastAPI e MySQL), mas reforçou a documentação técnica com artefactos que estavam ausentes. A principal decisão crítica foi separar o conceito de peça de catálogo do stock por loja, porque a mesma referência pode existir em várias filiais com quantidades diferentes. Também se validou que as operações de stock e faturação devem ser transacionais para evitar inconsistências.

Modelo: ChatGPT

Capítulo 3 — Implementação do Sistema

7. Implementação do Backend Aplicacional

7.1 Organização e Abordagem de Desenvolvimento

A implementação do backend cobriu a camada aplicacional do sistema DLMCare com FastAPI e Python 3.13. O trabalho abrangeu a definição do contrato da API, a modelação dos schemas de dados, o sistema de autenticação e controlo de acesso baseado em perfis, e o desenvolvimento de todos os módulos funcionais.

A arquitetura segue uma estrutura em camadas bem definida:

- **Schemas (DTOs)** — definição das estruturas de dados de entrada e saída com Pydantic v2, incluindo validações de negócio embutidas.
- **Routers** — camada de exposição HTTP, responsável pela receção dos pedidos, validação de autenticação/autorização e encaminhamento para os serviços.
- **Services** — camada de lógica de negócio, onde residem as regras do domínio, as validações de estado e os cálculos.
- **Utils** — funcionalidades auxiliares transversais: geração de PDF, envio de email e verificação de permissões partilhadas.

Para garantir desenvolvimento incremental e independência relativamente à base de dados (desenvolvida por outro elemento da equipa), os serviços foram inicialmente implementados com dados em memória (mocks), desenhados para integração posterior com repositórios reais sem alteração da lógica de negócio.

7.2 Contrato da API

Antes de qualquer implementação, foi definido um contrato formal da API no ficheiro docs/backend_api_contract_etapa3.md (aproximadamente 1600 linhas). Esta abordagem API-first permitiu que o frontend fosse desenvolvido em paralelo com o backend, eliminando ambiguidades entre as duas partes e servindo como referência vinculativa para ambas.

O contrato estabelece:

- Prefixo base: /api/v1 em todos os endpoints.
- Formato de resposta simples: {"data": {...}, "message": "..."}.
- Formato de resposta paginada: {"data": [...], "total": N, "page": 1, "page_size": 20, "pages": M}.
- Formato de erro normalizado: {"detail": "...", "code": "..."} com códigos semânticos como INVALID_CREDENTIALS, PERMISSION_DENIED, LOJA_MISMATCH, RESOURCE_NOT_FOUND, DUPLICATE_ENTRY, INVALID_STATE_TRANSITION, INSUFFICIENT_STOCK, entre outros.
- Documentação de cada endpoint: método, path, perfis autorizados, exemplos de request/response, erros possíveis e estado de implementação.

7.3 Schemas e Validações

Todos os contratos de dados foram modelados em Pydantic v2, no diretório `backend/app/schemas/`. Os schemas asseguram validação automática dos dados de entrada antes de atingirem a lógica de negócio. Foram criados 14 ficheiros de schemas, cobrindo todas as entidades do sistema.

Os enums definem os valores possíveis dos campos controlados, nomeadamente `PerfilUtilizador` (quatro perfis), `EstadoOrdemServico` (oito estados), `TipoEventoAuditoria` (29 eventos auditáveis), `CategoriaPeca`, `EstadoFatura` e `TipoMovimentoStock`, entre outros.

As validações embutidas nos schemas incluem:

- **NIF português:** exatamente 9 dígitos numéricos.
- **Telemóvel:** 9 dígitos, começando obrigatoriamente por 9.
- **Consentimento RGPD:** campo booleano obrigatoriamente `true` no registo de clientes — o valor `false` é rejeitado com HTTP 422 antes de chegar à lógica de negócio.
- **Valores monetários:** maior ou igual a 0.
- **preco_custo das peças nunca exposto:** o schema `PecaResponse` não inclui este campo. O `preco_custo` é armazenado internamente no serviço mas nunca serializado em nenhuma resposta pública da API.

7.4 Autenticação e Controlo de Acesso

O sistema de autenticação foi implementado com tokens JWT em `app/core/security.py`, com as dependências FastAPI correspondentes em `app/auth/dependencies.py`.

Tokens JWT: em cada autenticação são emitidos dois tokens — um access token (validade de 8 horas) e um refresh token de duração mais longa. O payload inclui `sub` (ID do utilizador), `nome`, `email`, `perfil`, `loja_id`, `loja_nome` e `ativo`. Toda a informação do utilizador viaja no token, pelo que nenhum acesso à base de dados é necessário para validar um pedido autenticado. A segurança das passwords utiliza `bcrypt` com custo 12, contornando uma incompatibilidade conhecida entre `passlib 1.7.4` e `bcrypt >= 5.x`.

Controlo de acesso por perfil (RBAC): a dependência `require_roles(*perfis)` é uma factory que gera dependências FastAPI reutilizáveis para cada endpoint. Uma tentativa de acesso sem o perfil adequado devolve 403 com código `PERMISSION_DENIED`.

Multi-tenancy por loja: todos os utilizadores com perfil diferente de `ADMINISTRADOR` têm um `loja_id` obrigatório no token. A dependência `get_loja_context()` garante que não-administradores apenas acedem aos dados da sua loja — o acesso a dados de outra loja resulta em 403 `LOJA_MISMATCH`. O `ADMINISTRADOR` tem `loja_id = null` e acesso global a todos os dados.

7.5 Módulos Funcionais

O backend expõe 14 módulos funcionais, cada um com o seu router e serviço correspondente, todos registados com o prefixo `/api/v1`:

Autenticação (`/auth`): `POST /login` devolve par de tokens; `POST /refresh` renova o par a partir de um refresh token válido; `GET /me` devolve o perfil do utilizador autenticado diretamente a partir do token, sem acesso à BD.

Clientes (`/clientes`): registo com validação de NIF único, telemóvel e consentimento RGPD; listagem com filtragem automática por loja para não-administradores; detalhe com trotinetes associadas e histórico de ordens de serviço; nível de fidelização e desconto sugerido calculados por cliente (RF16, descrito na secção 7.7).

Trotinetes (/trotinetes): associação obrigatória a um cliente; número de série único validado; filtragem por cliente.

Pecas (/pecas): catálogo global com categorização; distinção obrigatória entre preco_custo (interno) e preco_venda (exposto ao cliente); filtragem por categoria e pesquisa por nome/referência.

Stock (/stock): gestão de inventário por loja; entradas de stock; transferências diretas entre lojas; alertas automáticos quando quantidade <= limite_minimo; consumo automático de stock ao adicionar uma peça a uma OS.

Ordens de Serviço (/ordens-servico): módulo central, descrito em detalhe na secção 7.6.

Faturação (/faturas): emissão de faturas com cálculo de valor final, descontos comerciais e geração/envio de PDF; descrito na secção 7.7.

Dashboard (/dashboard): métricas calculadas dinamicamente — ordens por estado, ordens concluídas por loja, tempo médio de reparação no período, faturação total, peças abaixo do stock mínimo e eficiência por mecânico (ordens concluídas e minutos trabalhados totais, com base nos registos de tempo individuais).

Auditoria (/auditoria): registo de 29 tipos de eventos, cobrindo autenticação (sucesso e falha), ciclo de vida das OS, movimentações de stock, transferências, pedidos de peça, emissão de faturas e alterações a todas as entidades do sistema. Administradores veem todos os registos; gestores veem apenas os da sua loja.

Utilizadores (/utilizadores): gestão de staff pelo administrador — criação, edição de dados, desativação de contas e reposição de password; protegido exclusivamente para ADMINISTRADOR.

Lojas (/lojas): listagem e detalhe de lojas; administradores veem todas; restantes perfis veem apenas a sua loja.

Transferências e Pedidos de Peça (/transferencias, /pedidos-peca): fluxo de pedido de transferência entre lojas com aprovação do gerente da loja de origem, deduções e adições de stock automáticas, e geração de PDF do documento de transferência. Pedido de peça do mecânico ao gerente quando o stock está esgotado numa OS. Ambos os fluxos geram notificações em tempo real ao destinatário.

Catálogo de Serviços (/servicos): gestão do catálogo de serviços predefinidos, com nome e preço base, utilizados no diagnóstico das OS; criação e edição restritas a ADMINISTRADOR e GERENTE_LOJA.

Notificações (/notificacoes): inbox de notificações por utilizador; contagem de não lidas; marcação como lida individual ou em massa.

7.6 Máquina de Estados das Ordens de Serviço

As Ordens de Serviço constituem o núcleo operacional do sistema e seguem uma máquina de estados com 8 estados e transições controladas por perfil:

PENDENTE → EM_DIAGNOSTICO → EM_REPARACAO ↔ AGUARDA_PECAS → CONCLUIDA → FATURADA
Qualquer estado anterior a FATURADA pode transitar para CANCELADA.

As transições permitidas e os perfis autorizados a executá-las encontram-se definidos numa tabela _TRANSICOES no serviço. A tentativa de uma transição inválida devolve 409 INVALID_STATE_TRANSITION; a tentativa por um perfil não autorizado devolve 403 PERMISSION_DENIED.

Fluxo de diagnóstico: ao concluir o diagnóstico, o mecânico submete os serviços identificados a partir do catálogo (podendo adicionar entradas personalizadas). O sistema calcula automaticamente o `preco_servico` como soma dos preços dos serviços selecionados, transita a OS diretamente para `EM_REPARACAO` e envia automaticamente um email ao cliente com o resumo das operações a realizar.

Registo de tempos: cada sessão de trabalho ativa é registada com início, fim e minutos acumulados, associados ao mecânico responsável. Um mecânico não pode ter timers ativos em duas OS em simultâneo — uma tentativa devolve 409 `MECANICO_TIMER_CONFLICT` com os dados da OS conflituante, permitindo ao frontend apresentar um diálogo de confirmação para mudança de OS. A conclusão de uma OS requer timer ativo no momento da transição.

Observações internas: todos os perfis autenticados podem adicionar observações internas a uma OS. As transições de estado com texto de observação são guardadas automaticamente com um prefixo identificativo (ex.: [Conclusão da Reparação]), diferenciando-as de notas livres.

Sinalização de atraso (RF17): cada OS em curso é comparada contra o tempo médio de conclusão histórico. As que excedem essa média são sinalizadas com `em_atraso: true` e `minutos_em_atraso` calculados, permitindo filtragem no endpoint de listagem.

Reatribuição de mecânico: administradores e gestores podem reatribuir ou remover o mecânico de uma OS a qualquer momento. A reatribuição encerra automaticamente qualquer timer aberto pelo mecânico anterior, preservando os minutos acumulados.

7.7 Regras de Negócio

Regra de faturação: o valor final de uma fatura é calculado exclusivamente como:

$$\begin{aligned} \text{valor_final} &= \text{preco_servico} + \text{subtotal_pecas} \\ \text{onde subtotal_pecas} &= \sum(\text{quantidade} \times \text{preco_venda_unitario}) \end{aligned}$$

O `preco_custo` das peças nunca entra em nenhum cálculo exposto ao cliente. Esta invariante é garantida a dois níveis: o schema `PecaResponse` não inclui o campo, e o serviço de faturação usa exclusivamente o snapshot de `preco_venda_unitario` registado no momento em que a peça foi adicionada à OS — evitando discrepâncias caso o preço de venda seja alterado posteriormente.

Desconto de fidelização (RF16): o nível de fidelização do cliente é calculado automaticamente com a fórmula:

$$\text{nivel} = \min(5, \lfloor \log_2(n_{\text{concluidas}} + 1) \rfloor)$$

O nível máximo é 5, correspondendo a 10% de desconto (2% por nível). O desconto sugerido é apresentado no momento de emissão da fatura, podendo ser ajustado manualmente. Suporta descontos percentuais e fixos.

Unicidade de faturas: uma OS só pode ter uma fatura associada. A tentativa de emitir uma segunda fatura para a mesma OS resulta em 409 `ORDER_ALREADY_INVOICED`, verificado antes da validação de estado para cobrir o caso em que a OS já transitou para `FATURADA`.

7.8 Funcionalidades Auxiliares

Geração de PDF (`utils/pdf.py`): faturas e documentos de transferência são gerados em PDF com `fpdf2` 2.8.3, incluindo cabeçalho com identidade da loja, dados do cliente e trotinete, tabela de peças e serviços, e secção de totais com desconto quando aplicável.

Envio de email (`utils/email.py`): notificações automáticas ao cliente em português com templates HTML para quatro eventos — trotinete pronta para levantamento, OS cancelada (RF14), resumo do diagnóstico, e fatura com PDF em anexo. O envio utiliza `BackgroundTasks` do FastAPI para não bloquear a resposta HTTP. Falhas de SMTP são silenciosas, não afetando o resultado da operação principal.

Tratamento global de erros: um exception handler registrado em `main.py` converte os erros de validação Pydantic (HTTP 422) para o formato de erro normalizado do contrato, garantindo consistência em todas as respostas de erro da API.